```
import pandas as pd

df = pd.read_csv('WA_Fn-UseC_-HR-Employee-Attrition 3.csv')

# ── 1. Basic shape ──────────────────────────────────────
print("Rows, Columns:", df.shape)

# ── 2. Column names + data types ────────────────────────
print("\nColumn Info:")
print(df.dtypes)

# ── 3. First look ───────────────────────────────────────
df.head()
```

```
Rows, Columns: (1470, 35)

Column Info:
Age                         int64
Attrition                  object
BusinessTravel             object
DailyRate                   int64
Department                 object
DistanceFromHome            int64
Education                   int64
EducationField             object
EmployeeCount               int64
EmployeeNumber              int64
EnvironmentSatisfaction     int64
Gender                     object
HourlyRate                  int64
JobInvolvement              int64
JobLevel                    int64
JobRole                    object
JobSatisfaction             int64
MaritalStatus              object
MonthlyIncome               int64
MonthlyRate                 int64
NumCompaniesWorked          int64
Over18                     object
OverTime                   object
PercentSalaryHike           int64
PerformanceRating           int64
RelationshipSatisfaction    int64
StandardHours               int64
StockOptionLevel            int64
TotalWorkingYears           int64
TrainingTimesLastYear       int64
WorkLifeBalance             int64
YearsAtCompany              int64
YearsInCurrentRole          int64
YearsSinceLastPromotion     int64
YearsWithCurrManager        int64
dtype: object
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	Educatio
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life S
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life S
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life S
4	27	No	Travel_Rarely	591	Research & Development	2	1	

5 rows × 35 columns

```
# ── 4. Statistical summary (numeric columns) ──────────
df.describe().round(2)
```

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumber	Environme
count	1470.00	1470.00	1470.00	1470.00	1470.0	1470.00	
mean	36.92	802.49	9.19	2.91	1.0	1024.87	
std	9.14	403.51	8.11	1.02	0.0	602.02	
min	18.00	102.00	1.00	1.00	1.0	1.00	
25%	30.00	465.00	2.00	2.00	1.0	491.25	
50%	36.00	802.00	7.00	3.00	1.0	1020.50	
75%	43.00	1157.00	14.00	4.00	1.0	1555.75	
max	60.00	1499.00	29.00	5.00	1.0	2068.00	

8 rows × 26 columns

```
# ── 5. Missing values check ──────────────────────────
missing = df.isnull().sum()
print("Missing values:\n", missing[missing > 0])
print("\nTotal missing:", df.isnull().sum().sum())
```

```
Missing values:
 Series([], dtype: int64)

Total missing: 0
```

```
# ── 6. Duplicate rows ────────────────────────────────
print("Duplicate rows:", df.duplicated().sum())
```

```
Duplicate rows: 0
```

```
# ── 7. Key columns distribution ──────────────────────
print("Gender:\n", df['Gender'].value_counts())
print("\nDepartment:\n", df['Department'].value_counts())
print("\nJobLevel:\n", df['JobLevel'].value_counts().sort_index())
print("\nAttrition:\n", df['Attrition'].value_counts())
```

```
Gender:
 Gender
Male      882
Female    588
Name: count, dtype: int64

Department:
 Department
Research & Development    961
Sales                     446
Human Resources            63
Name: count, dtype: int64

JobLevel:
 JobLevel
1    543
2    534
3    218
4    106
5     69
Name: count, dtype: int64

Attrition:
 Attrition
No     1233
Yes     237
Name: count, dtype: int64
```

```
#Bring Department to second column ──────────────
cols = df.columns.tolist()
cols.insert(1, cols.pop(cols.index('Department')))
df = df[cols]

df.head(3)
```

	Age	Department	Attrition	BusinessTravel	DailyRate	DistanceFromHome	Education	Educatio
0	41	Sales	Yes	Travel_Rarely	1102	1	2	Life S
1	49	Research & Development	No	Travel_Frequently	279	8	1	Life S
2	37	Research & Development	Yes	Travel_Rarely	1373	2	2	

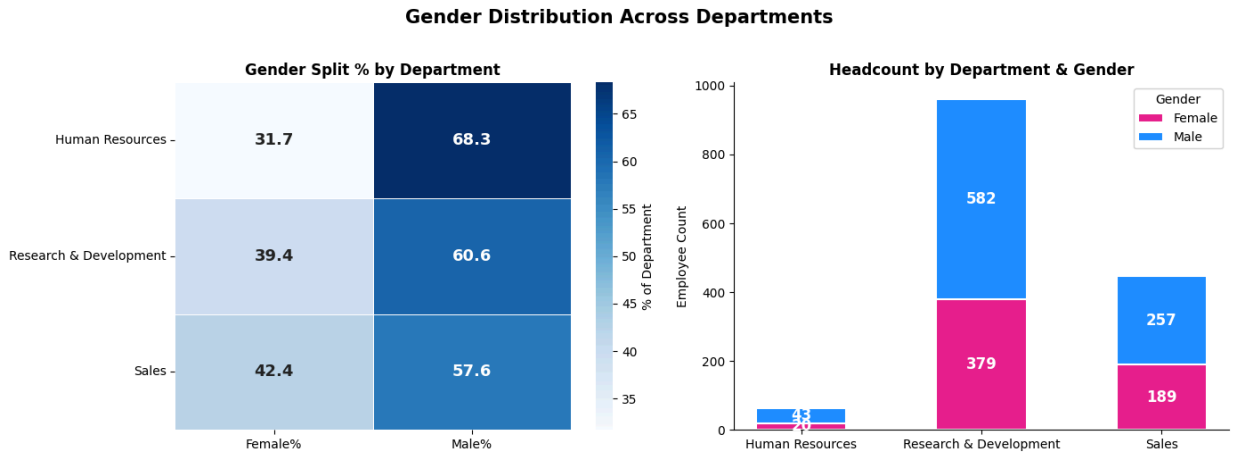
3 rows × 35 columns

```
#Gender distribution across departments ──────────
gender_dept = df.groupby(['Department', 'Gender']).size().unstack()
gender_dept['Total'] = gender_dept.sum(axis=1)
gender_dept['Female%'] = (gender_dept['Female'] / gender_dept['Total'] * 100).round(1)
gender_dept['Male%'] = (gender_dept['Male'] / gender_dept['Total'] * 100).round(1)

print(gender_dept)
```

```
Gender                  Female  Male  Total  Female%  Male%
Department
Human Resources             20    43     63     31.7   68.3
Research & Development     379   582    961     39.4   60.6
Sales                      189   257    446     42.4   57.6
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# ── Data prep ──────────────────────────────────────────────
gender_dept = df.groupby(['Department', 'Gender']).size().unstack()
gender_dept['Total'] = gender_dept.sum(axis=1)
gender_dept['Female%'] = (gender_dept['Female'] / gender_dept['Total'] * 100).round(1)
gender_dept['Male%'] = (gender_dept['Male'] / gender_dept['Total'] * 100).round(1)

# ── Plot ───────────────────────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Gender Distribution Across Departments', fontsize=15, fontweight='bold', y=1.0

# Chart 1 — Heatmap (% breakdown)
heat_data = gender_dept[['Female%', 'Male%']]
sns.heatmap(
    heat_data,
    annot=True,
    fmt='.1f',
    cmap='Blues',
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': '% of Department'},
    ax=axes[0],
    annot_kws={"size": 13, "weight": "bold"}
)
axes[0].set_title('Gender Split % by Department', fontweight='bold')
axes[0].set_xlabel('')
axes[0].set_ylabel('')
axes[0].tick_params(axis='x', rotation=0)

# Chart 2 — Stacked bar (actual headcount)
colors = ['#E91E8C', '#1E90FF']
bottom = np.zeros(len(gender_dept))
```

```
for i, (gender, color) in enumerate(zip(['Female', 'Male'], colors)):
    bars = axes[1].bar(
        gender_dept.index,
        gender_dept[gender],
        bottom=bottom,
        color=color,
        label=gender,
        width=0.5,
        edgecolor='white',
        linewidth=1.5
    )
    # Add count labels inside bars
    for bar, val in zip(bars, gender_dept[gender]):
        axes[1].text(
            bar.get_x() + bar.get_width()/2,
            bottom[list(gender_dept.index).index(bar.get_x() + bar.get_width()/2 + bar.get_x
            str(val),
            ha='center', va='center',
            fontsize=12, fontweight='bold', color='white'
        )
    bottom += gender_dept[gender].values

axes[1].set_title('Headcount by Department & Gender', fontweight='bold')
axes[1].set_xlabel('')
axes[1].set_ylabel('Employee Count')
axes[1].legend(title='Gender', loc='upper right')
axes[1].spines[['top', 'right']].set_visible(False)
axes[1].tick_params(axis='x', rotation=0)

plt.tight_layout()
plt.savefig('gender_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
```




```
Find top quartile threshold per department ———
q75 = df.groupby('Department')['MonthlyIncome'].quantile(0.75)
print("75th percentile cutoff per department:\n", q75.round(2))
```

```
75th percentile cutoff per department:
 Department
Human Resources           8412.50
Research & Development    7655.00
Sales                     8776.75
Name: MonthlyIncome, dtype: float64
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from scipy import stats

# —— 1. Quartile boundaries ————————————————————
q25 = df['MonthlyIncome'].quantile(0.25)
```

```
q50 = df['MonthlyIncome'].quantile(0.50)
q75 = df['MonthlyIncome'].quantile(0.75)
mean = df['MonthlyIncome'].mean()
std = df['MonthlyIncome'].std()
skewness = df['MonthlyIncome'].skew()

print("═══ MonthlyIncome Summary ═══")
print(f"Bottom Quartile (Q1) cutoff : {q25:,.0f}")
print(f"Median          (Q2) cutoff : {q50:,.0f}")
print(f"Top Quartile    (Q3) cutoff : {q75:,.0f}")
print(f"Mean                        : {mean:,.0f}")
print(f"Std Deviation               : {std:,.0f}")
print(f"Skewness                    : {skewness:.3f}")
print()
if skewness > 0.5:
    print("📊 RIGHT SKEWED — most employees earn below mean, few high earners pulling it up"
elif skewness < -0.5:
    print("📊 LEFT SKEWED — most employees earn above mean")
else:
    print("📊 ROUGHLY SYMMETRIC — fairly balanced distribution")
```

```
═══ MonthlyIncome Summary ═══
Bottom Quartile (Q1) cutoff : 2,911
Median          (Q2) cutoff : 4,919
Top Quartile    (Q3) cutoff : 8,379
Mean                        : 6,503
Std Deviation               : 4,708
Skewness                    : 1.370

📊 RIGHT SKEWED — most employees earn below mean, few high earners pulling it up
```

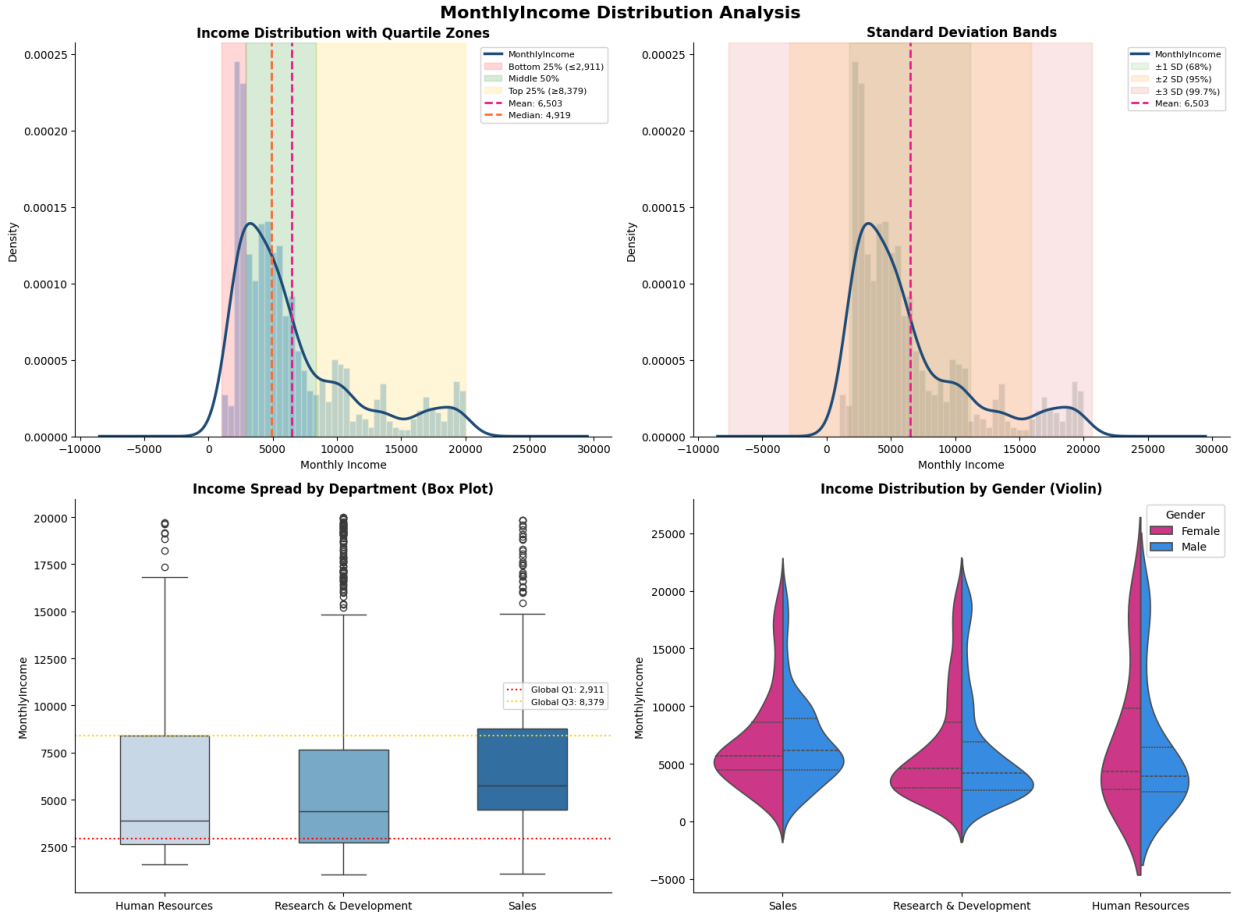
```
#Bottom quartile employees ──────────────────────
bottom_q = df[df['MonthlyIncome'] <= q25]
bq_gender = bottom_q.groupby(['Department', 'Gender']).size().unstack(fill_value=0)
bq_avg = bottom_q.groupby(['Department', 'Gender'])['MonthlyIncome'].mean().unstack().round(

print("Bottom Quartile Headcount:\n", bq_gender)
print("\nBottom Quartile Avg Salary:\n", bq_avg)
```

```
Bottom Quartile Headcount:
 Gender                  Female  Male
Department
Human Resources              7    16
Research & Development      99   183
Sales                       29    35

Bottom Quartile Avg Salary:
 Gender                   Female     Male
Department
Human Resources         2424.29  2349.00
Research & Development  2377.93  2355.25
Sales                   2364.86  2260.40
```

```
The full distribution chart ────────────────────
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle('MonthlyIncome Distribution Analysis', fontsize=16, fontweight='bold')

# ── Chart 1: Histogram + KDE with quartile bands ──────
ax = axes[0, 0]
ax.hist(df['MonthlyIncome'], bins=40, color='#4A90D9', alpha=0.4, edgecolor='white', density
df['MonthlyIncome'].plot.kde(ax=ax, color='#1F4E79', linewidth=2.5)

# Shade quartile zones
ax.axvspan(df['MonthlyIncome'].min(), q25, alpha=0.15, color='red',    label=f'Bottom 25% (≤
ax.axvspan(q25, q75,                       alpha=0.15, color='green', label=f'Middle 50%')
ax.axvspan(q75, df['MonthlyIncome'].max(),   alpha=0.15, color='gold',  label=f'Top 25% (≥{q

ax.axvline(mean, color='#E91E8C', linestyle='--', linewidth=2, label=f'Mean: {mean:,.0f}')
ax.axvline(q50,  color='#FF6B35', linestyle='--', linewidth=2, label=f'Median: {q50:,.0f}')
```

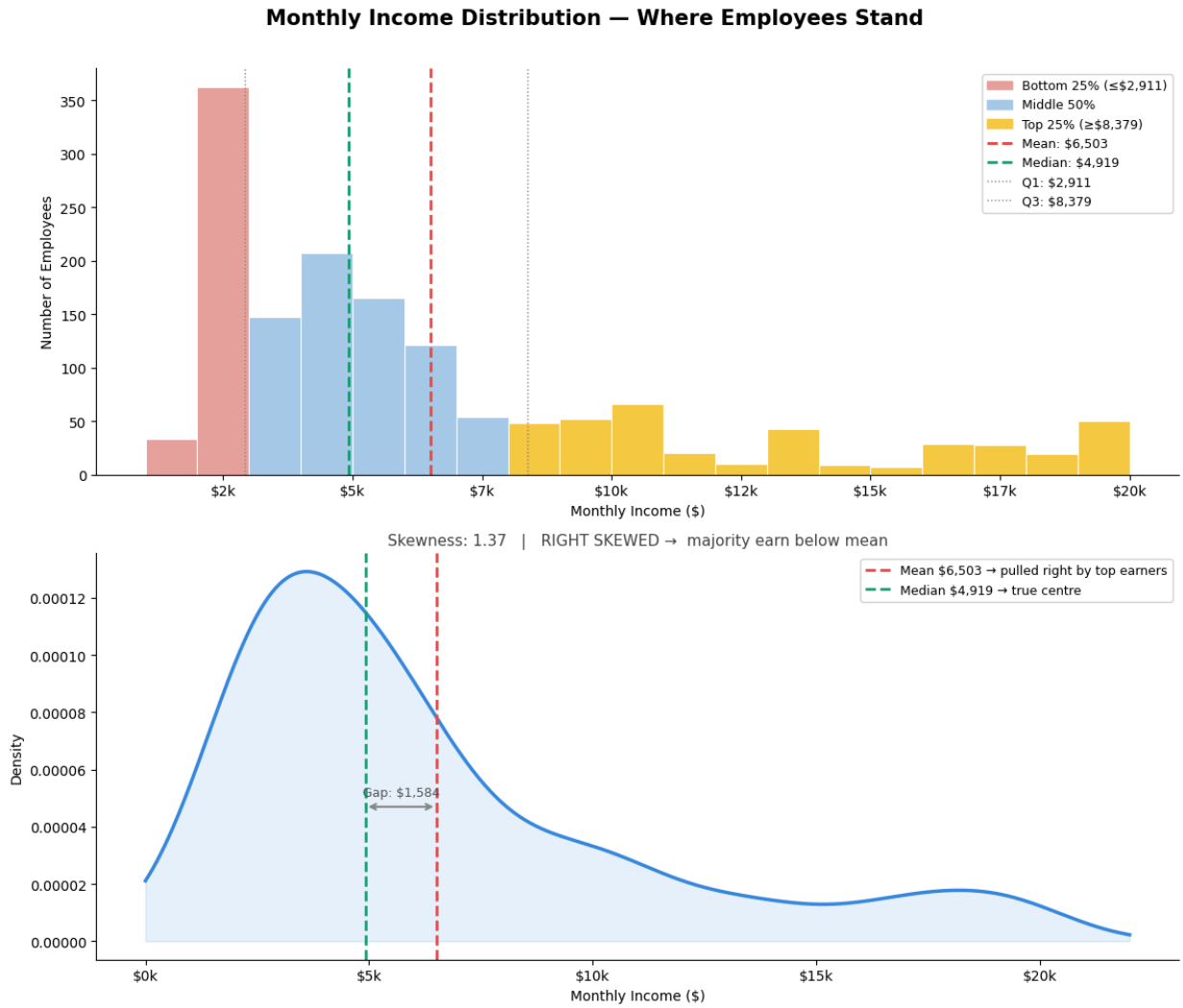
```
ax.set_title('Income Distribution with Quartile Zones', fontweight='bold')
ax.set_xlabel('Monthly Income')
ax.set_ylabel('Density')
ax.legend(fontsize=8)
ax.spines[['top', 'right']].set_visible(False)

# ── Chart 2: Std deviation bands ──────────────────────
ax2 = axes[0, 1]
ax2.hist(df['MonthlyIncome'], bins=40, color='#B0BEC5', alpha=0.5, edgecolor='white', densit
df['MonthlyIncome'].plot.kde(ax=ax2, color='#1F4E79', linewidth=2.5)

for i, (mult, color, label) in enumerate([
    (1, '#4CAF50', '±1 SD (68%)'),
    (2, '#FF9800', '±2 SD (95%)'),
    (3, '#F44336', '±3 SD (99.7%)'),
]):
    ax2.axvspan(mean - mult*std, mean + mult*std, alpha=0.12, color=color, label=label)

ax2.axvline(mean, color='#E91E8C', linestyle='--', linewidth=2, label=f'Mean: {mean:,.0f}')
ax2.set_title('Standard Deviation Bands', fontweight='bold')
ax2.set_xlabel('Monthly Income')
ax2.set_ylabel('Density')
ax2.legend(fontsize=8)
ax2.spines[['top', 'right']].set_visible(False)

# ── Chart 3: Box plot by Department ───────────────────
ax3 = axes[1, 0]
dept_order = df.groupby('Department')['MonthlyIncome'].median().sort_values().index
sns.boxplot(
    data=df, x='Department', y='MonthlyIncome',
    order=dept_order, palette='Blues',
    width=0.5, ax=ax3
)
ax3.axhline(q25, color='red',  linestyle=':', linewidth=1.5, label=f'Global Q1: {q25:,.0f}')
ax3.axhline(q75, color='gold', linestyle=':', linewidth=1.5, label=f'Global Q3: {q75:,.0f}')
ax3.set_title('Income Spread by Department (Box Plot)', fontweight='bold')
ax3.set_xlabel('')
ax3.legend(fontsize=8)
ax3.spines[['top', 'right']].set_visible(False)

# ── Chart 4: Violin by Gender ─────────────────────────
ax4 = axes[1, 1]
sns.violinplot(
    data=df, x='Department', y='MonthlyIncome',
    hue='Gender', split=True,
    palette={'Female': '#E91E8C', 'Male': '#1E90FF'},
    inner='quartile', ax=ax4
)
ax4.set_title('Income Distribution by Gender (Violin)', fontweight='bold')
ax4.set_xlabel('')
ax4.spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.savefig('income_distribution_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
/tmp/ipykernel_6315/2266164125.py:46: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assig

  sns.boxplot(
```




```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10))
fig.suptitle('Monthly Income Distribution — Where Employees Stand',
             fontsize=15, fontweight='bold', y=1.01)

q1  = df['MonthlyIncome'].quantile(0.25)
q3  = df['MonthlyIncome'].quantile(0.75)
med = df['MonthlyIncome'].median()
avg = df['MonthlyIncome'].mean()
skw = df['MonthlyIncome'].skew()

# ── Chart 1: Histogram with quartile zones ──────────────
bins = np.arange(1000, 21000, 1000)
n, edges, patches = ax1.hist(df['MonthlyIncome'], bins=bins, edgecolor='white', linewidth=0.

for patch, left in zip(patches, edges[:-1]):
    mid = left + 500
    if mid <= q1:
        patch.set_facecolor('#e8a49c')
    elif mid <= q3:
        patch.set_facecolor('#a8c8e8')
    else:
        patch.set_facecolor('#f5c842')

ax1.axvline(avg, color='#E24B4A', linewidth=2, linestyle='--', label=f'Mean: ${avg:,.0f}')
ax1.axvline(med, color='#1D9E75', linewidth=2, linestyle='--', label=f'Median: ${med:,.0f}')
```

```
ax1.axvline(q1,  color='#888',    linewidth=1, linestyle=':',  label=f'Q1: ${q1:,.0f}')
ax1.axvline(q3,  color='#888',    linewidth=1, linestyle=':',  label=f'Q3: ${q3:,.0f}')

legend_patches = [
    mpatches.Patch(color='#e8a49c', label=f'Bottom 25% (≤${q1:,.0f})'),
    mpatches.Patch(color='#a8c8e8', label='Middle 50%'),
    mpatches.Patch(color='#f5c842', label=f'Top 25% (≥${q3:,.0f})'),
]
ax1.legend(handles=legend_patches + ax1.get_lines()[:4], fontsize=9, loc='upper right')
ax1.set_xlabel('Monthly Income ($)')
ax1.set_ylabel('Number of Employees')
ax1.spines[['top','right']].set_visible(False)
ax1.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'${int(x/1000)}k'))

# ── Chart 2: KDE skewness curve ──────────────────────────
from scipy.stats import gaussian_kde
kde = gaussian_kde(df['MonthlyIncome'], bw_method=0.3)
x_range = np.linspace(0, 22000, 500)
ax2.plot(x_range, kde(x_range), color='#378ADD', linewidth=2.5)
ax2.fill_between(x_range, kde(x_range), alpha=0.12, color='#378ADD')

ax2.axvline(avg, color='#E24B4A', linewidth=2, linestyle='--', label=f'Mean ${avg:,.0f} → pu
ax2.axvline(med, color='#1D9E75', linewidth=2, linestyle='--', label=f'Median ${med:,.0f} →

gap = avg - med
ax2.annotate('', xy=(avg, kde(avg)[0]*0.6), xytext=(med, kde(avg)[0]*0.6),
             arrowprops=dict(arrowstyle='<->', color='#888', lw=1.5))
ax2.text((avg+med)/2, kde(avg)[0]*0.65, f'Gap: ${gap:,.0f}', ha='center', fontsize=9, color=

direction = "RIGHT SKEWED →  majority earn below mean" if skw > 0.5 else "LEFT SKEWED" if sk
ax2.set_title(f'Skewness: {skw:.2f}   |   {direction}', fontsize=11, color='#444')
ax2.legend(fontsize=9)
ax2.set_xlabel('Monthly Income ($)')
ax2.set_ylabel('Density')
ax2.spines[['top','right']].set_visible(False)
ax2.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f'${int(x/1000)}k'))

plt.tight_layout()
plt.savefig('income_distribution_skewness.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"\nSkewness: {skw:.3f}")
print("Interpretation: Mean > Median → RIGHT SKEW → most employees earn BELOW the average")
print(f"Mean-Median gap: ${gap:,.0f} — this is the 'illusion of average salary'")
```




```
Skewness: 1.370
Interpretation: Mean > Median → RIGHT SKEW → most employees earn BELOW the average
Mean-Median gap: $1,584 — this is the 'illusion of average salary'
```

```
df.describe().round(2)
```

Age	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumber	Environmer
470.00	1470.00	1470.00	1470.00	1470.0	1470.00	
36.92	802.49	9.19	2.91	1.0	1024.87	
9.14	403.51	8.11	1.02	0.0	602.02	
18.00	102.00	1.00	1.00	1.0	1.00	
30.00	465.00	2.00	2.00	1.0	491.25	
36.00	802.00	7.00	3.00	1.0	1020.50	
43.00	1157.00	14.00	4.00	1.0	1555.75	
60.00	1499.00	29.00	5.00	1.0	2068.00	

5 columns

```
df.head()
```

```
df['CompaRatio'] = df['MonthlyIncome'] / df['Midpoint']

df['BandPosition'] = np.where(df['CompaRatio'] < 0.8, 'Below Band',
                      np.where(df['CompaRatio'] > 1.2, 'Above Band', 'Within Band'))
```

```
df.describe().round(2)
```

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumber	Environme
count	1470.00	1470.00	1470.00	1470.00	1470.0	1470.00	
mean	36.92	802.49	9.19	2.91	1.0	1024.87	
std	9.14	403.51	8.11	1.02	0.0	602.02	
min	18.00	102.00	1.00	1.00	1.0	1.00	
25%	30.00	465.00	2.00	2.00	1.0	491.25	
50%	36.00	802.00	7.00	3.00	1.0	1020.50	
75%	43.00	1157.00	14.00	4.00	1.0	1555.75	
max	60.00	1499.00	29.00	5.00	1.0	2068.00	

8 rows × 28 columns

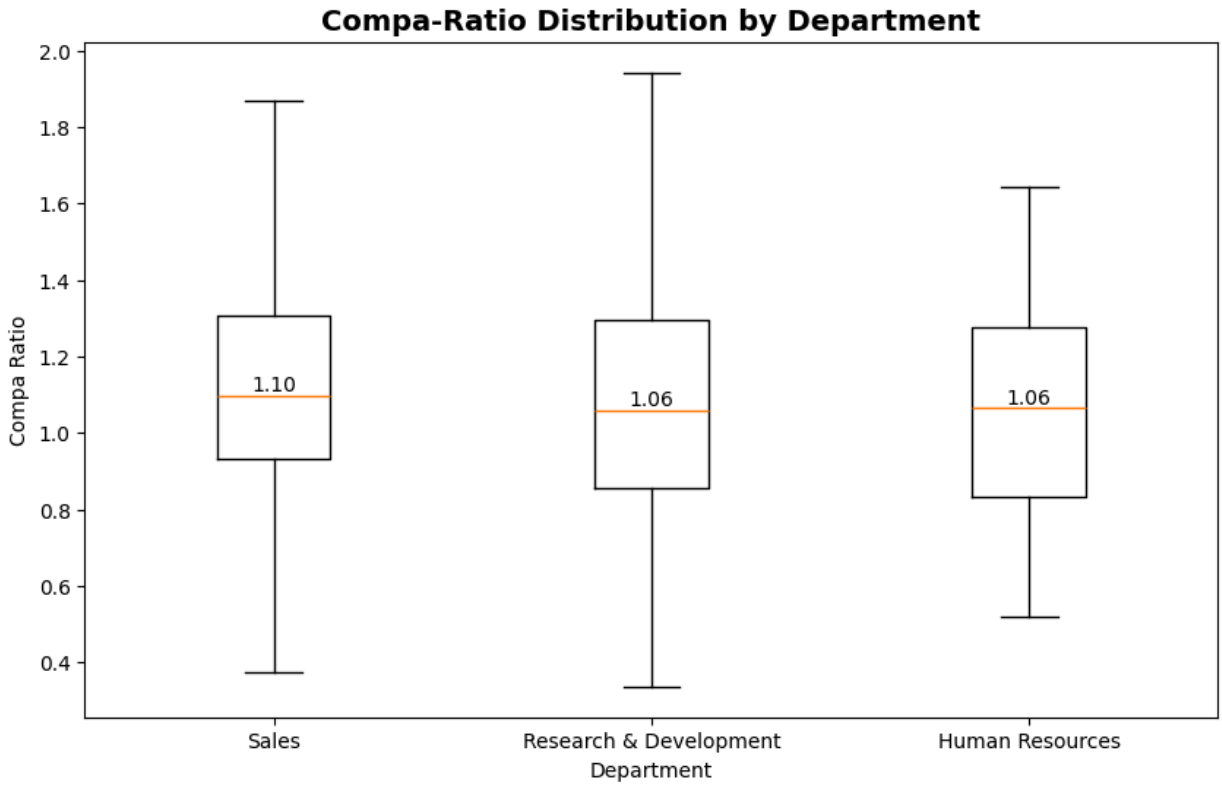
```
outliers = df[df['BandPosition'] != 'Within Band']

outlier_summary = (
    outliers
    .groupby(['Department', 'JobLevel', 'BandPosition'])
    .size()
    .unstack(fill_value=0)
)

print(outlier_summary)
```

```
BandPosition                     Above Band  Below Band
Department             JobLevel
Human Resources        1                 3          14
                       2                 6           1
                       3                 4           0
                       4                 4           0
                       5                 7           0
Research & Development 1                74         138
                       2                75          28
                       3                89           2
                       4                62           0
                       5                49           0
Sales                  1                 3          32
                       2                84           4
                       3                47           0
```

```
4                  29           0
5                  13           0
```

```
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(10,6))

# Prepare data
departments = df['Department'].unique()
data = [df[df['Department']==d]['CompaRatio'] for d in departments]

# Boxplot (best for comp analysis)
ax.boxplot(data, labels=departments, showfliers=False)

# Median line
medians = [np.median(d) for d in data]
for i, m in enumerate(medians, start=1):
    ax.text(i, m, f'{m:.2f}', ha='center', va='bottom')

ax.set_title('Compa-Ratio Distribution by Department', fontsize=14, fontweight='bold')
ax.set_ylabel('Compa Ratio')
ax.set_xlabel('Department')

plt.show()
```

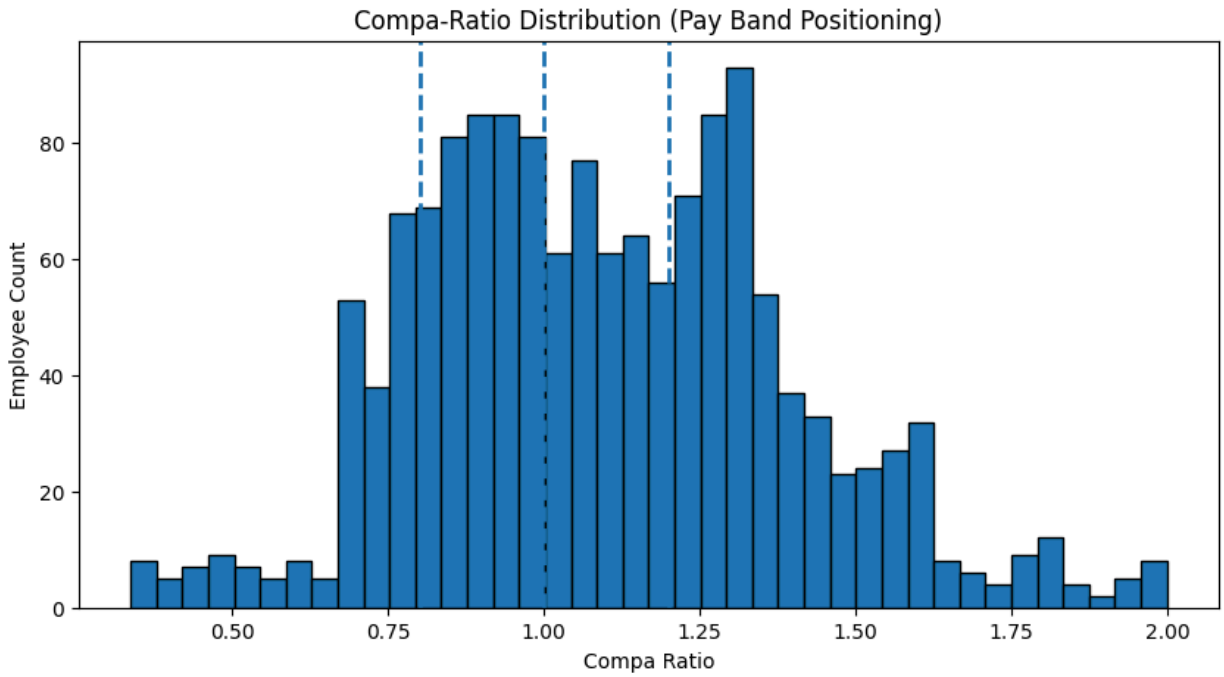
```
/tmp/ipykernel_6315/2573303624.py:11: MatplotlibDeprecationWarning: The 'labels' parameter of
  ax.boxplot(data, labels=departments, showfliers=False)
```



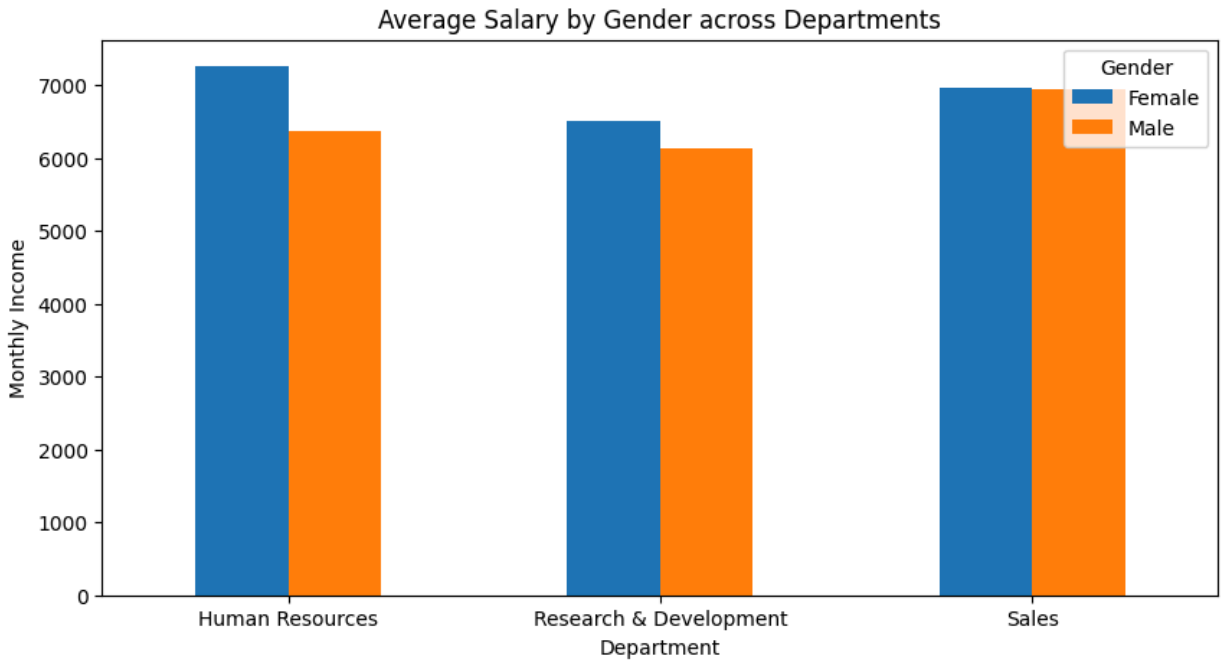

```
gender_pay = (
    df.groupby(['Department', 'Gender'])['MonthlyIncome']
    .mean()
    .unstack()
)

gender_pay['PayGap%'] = ((gender_pay['Male'] - gender_pay['Female']) / gender_pay['Male']) *
gender_pay = gender_pay.round(2)

print(gender_pay)
```

```
Gender                   Female     Male  PayGap%
Department
Human Resources         7264.00  6371.02   -14.02
Research & Development  6513.69  6129.89    -6.26
Sales                   6972.13  6949.65    -0.32
```

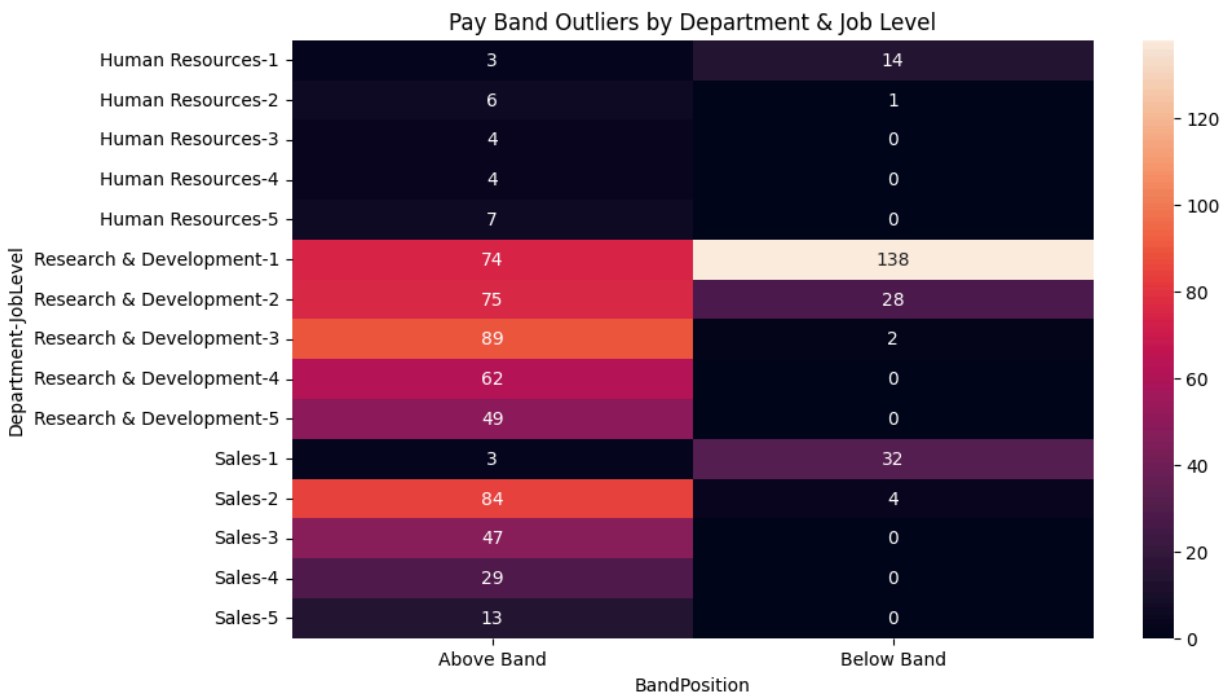
```
plt.figure(figsize=(10,5))

plt.hist(df['CompaRatio'], bins=40, edgecolor='black')

plt.axvline(0.8, linestyle='--', linewidth=2)
plt.axvline(1.0, linestyle='--', linewidth=2)
plt.axvline(1.2, linestyle='--', linewidth=2)

plt.title('Compa-Ratio Distribution (Pay Band Positioning)')
plt.xlabel('Compa Ratio')
plt.ylabel('Employee Count')

plt.show()
```




```
gender_pay[['Female', 'Male']].plot(kind='bar', figsize=(10,5))

plt.title('Average Salary by Gender across Departments')
plt.ylabel('Monthly Income')
plt.xticks(rotation=0)

plt.show()
```




```
import seaborn as sns

plt.figure(figsize=(10,6))

sns.heatmap(outlier_summary, annot=True, fmt='d')

plt.title('Pay Band Outliers by Department & Job Level')

plt.show()
```




```
print("Total Employees:", len(df))
print("Below Band:", (df['BandPosition']=='Below Band').sum())
print("Above Band:", (df['BandPosition']=='Above Band').sum())

# Absolute gap (magnitude)
gender_pay['AbsGap'] = gender_pay['PayGap%'].abs()
```

```
# Now find highest gap correctly
highest_gap = gender_pay['AbsGap'].sort_values(ascending=False).head(1)

print("Highest Pay Gap Department (by magnitude):")
print(highest_gap)
```

```
Total Employees: 1470
Below Band: 219
Above Band: 549
Highest Pay Gap Department (by magnitude):
Department
Human Resources    14.02
Name: AbsGap, dtype: float64
```

```
adj_gap = (
    df.groupby(['Department', 'JobLevel', 'Gender'])['MonthlyIncome']
    .mean()
    .unstack()
)

adj_gap['Gap%'] = ((adj_gap['Male'] - adj_gap['Female']) / adj_gap['Male']) * 100
```

```
# Find worst outlier zone
worst = outlier_summary.sum(axis=1).sort_values(ascending=False).head(3)
worst
```

		0
Department	**JobLevel**	
Research & Development	**1**	212
	2	103
	3	91

dtype: int64

Start coding or generate with AI.

		0
Department	**JobLevel**	
Research & Development	**1**	212
	2	103
	3	91

dtype: int64

```
# Data validation checks
checks = {
    "Missing Salary": df['MonthlyIncome'].isnull().sum(),
    "Zero Salary": (df['MonthlyIncome'] == 0).sum(),
    "Invalid Compa Ratio": ((df['CompaRatio'] < 0.5) | (df['CompaRatio'] > 2)).sum()
}

checks
```

```
{'Missing Salary': np.int64(0),
 'Zero Salary': np.int64(0),
 'Invalid Compa Ratio': np.int64(29)}
```

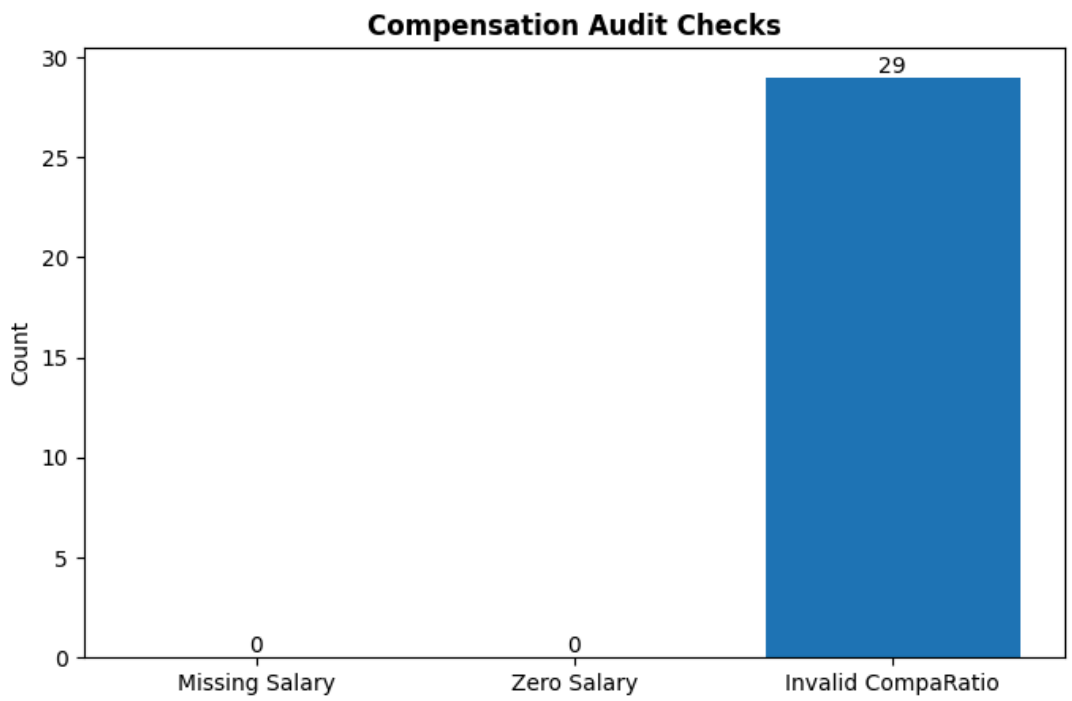
```
# simulate salary correction
df['AdjustedSalary'] = df['MonthlyIncome']
```

```
# bring below-band employees to minimum threshold
df.loc[df['CompaRatio'] < 0.8, 'AdjustedSalary'] = df['Midpoint'] * 0.8
```

```
# simple bonus logic
df['Bonus'] = np.where(df['PerformanceRating'] >= 4,
                       df['MonthlyIncome'] * 0.15,
                       df['MonthlyIncome'] * 0.05)
```

```
risk_summary = pd.DataFrame({
    'Metric': ['Below Band', 'Above Band', 'High Gap Dept'],
    'Value': [
        (df['BandPosition']=='Below Band').sum(),
        (df['BandPosition']=='Above Band').sum(),
        gender_pay['AbsGap'].idxmax()
    ]
})
```

```
import matplotlib.pyplot as plt

# Audit metrics
metrics = {
    "Missing Salary": df['MonthlyIncome'].isnull().sum(),
    "Zero Salary": (df['MonthlyIncome'] == 0).sum(),
    "Invalid CompaRatio": ((df['CompaRatio'] < 0.5) | (df['CompaRatio'] > 2)).sum()
}

fig, ax = plt.subplots(figsize=(8,5))

labels = list(metrics.keys())
values = list(metrics.values())

bars = ax.bar(labels, values)

for bar in bars:
    y = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, y, f'{int(y)}',
            ha='center', va='bottom')

ax.set_title('Compensation Audit Checks', fontweight='bold')
ax.set_ylabel('Count')

plt.show()
```



```
invalid_compa = df[(df['CompaRatio'] < 0.5) | (df['CompaRatio'] > 2)]

invalid_compa.head(10)
```

	Age	Department	Attrition	BusinessTravel	DailyRate	DistanceFromHome	Education	Educat
23	21	Research & Development	No	Travel_Rarely	391	15	2	Lif
33	39	Sales	Yes	Travel_Rarely	895	5	3	Techni
61	38	Research & Development	No	Travel_Frequently	653	29	5	Lif
99	44	Research & Development	No	Non-Travel	489	23	3	
149	19	Research & Development	No	Travel_Rarely	1181	3	1	
152	53	Sales	No	Travel_Rarely	1436	6	2	
153	45	Research & Development	No	Travel_Rarely	194	9	3	Lif
177	19	Research & Development	Yes	Travel_Rarely	303	2	3	Lif
262	32	Research & Development	No	Travel_Rarely	128	2	1	Techni
296	18	Research & Development	Yes	Travel_Rarely	230	3	3	Lif

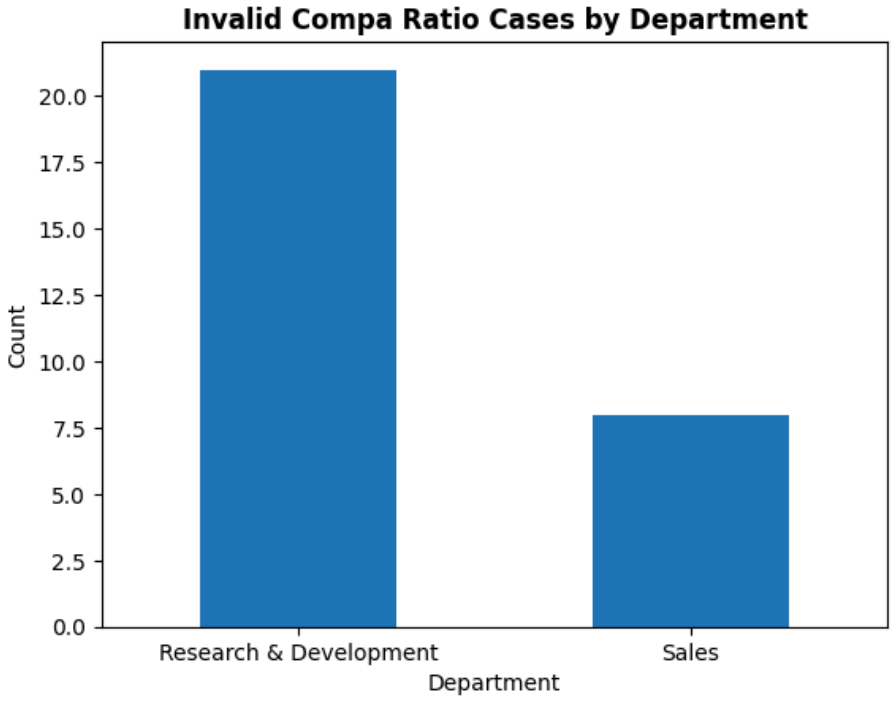
10 rows × 40 columns

```
invalid_compa[['EmployeeNumber','Department','JobLevel',
               'MonthlyIncome','Midpoint','CompaRatio','BandPosition']].head(15)
```

	EmployeeNumber	Department	JobLevel	MonthlyIncome	Midpoint	CompaRatio	BandPosition
23	30	Research & Development	1	1232	3000	0.410667	Below Band
33	42	Sales	2	2086	5000	0.417200	Below Band
61	79	Research & Development	2	2406	5000	0.481200	Below Band
99	132	Research & Development	2	2042	5000	0.408400	Below Band
149	201	Research & Development	1	1483	3000	0.494333	Below Band
152	205	Sales	2	2306	5000	0.461200	Below Band
153	206	Research & Development	2	2348	5000	0.469600	Below Band
177	243	Research & Development	1	1102	3000	0.367333	Below Band
262	362	Research & Development	2	2176	5000	0.435200	Below Band
296	405	Research & Development	1	1420	3000	0.473333	Below Band
301	411	Sales	1	1200	3000	0.400000	Below Band
513	701	Research & Development	1	1009	3000	0.336333	Below Band
515	704	Research & Development	1	1281	3000	0.427000	Below Band
556	769	Research & Development	2	2450	5000	0.490000	Below Band
727	1012	Research & Development	1	1051	3000	0.350333	Below Band

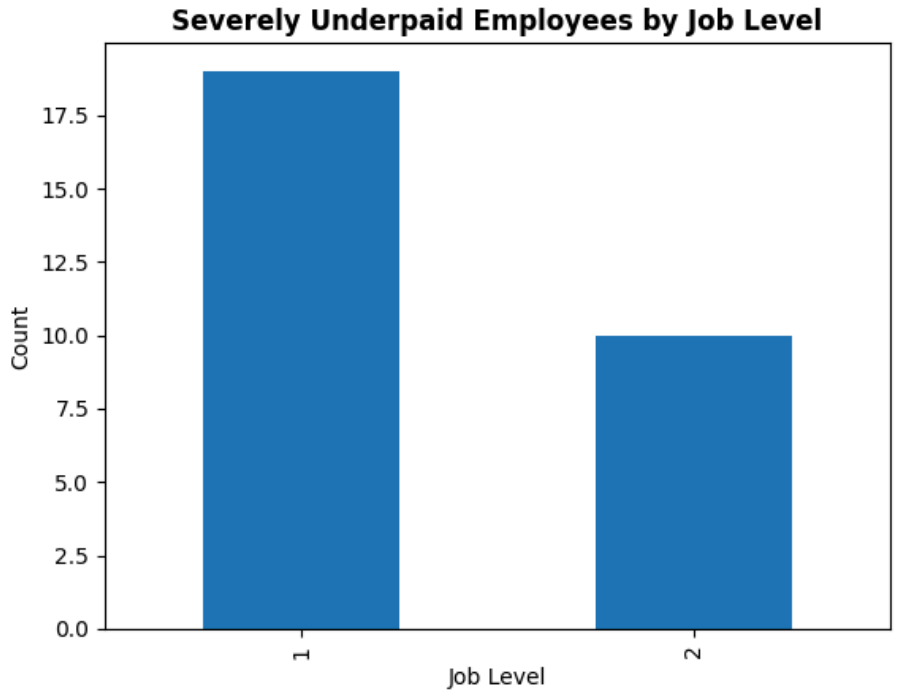
```
invalid_compa.sort_values(by='CompaRatio', ascending=False).head(10)
```

```
import matplotlib.pyplot as plt

invalid_compa.groupby('Department').size().plot(kind='bar')

plt.title('Invalid Compa Ratio Cases by Department', fontweight='bold')
plt.ylabel('Count')
plt.xticks(rotation=0)

plt.show()
```

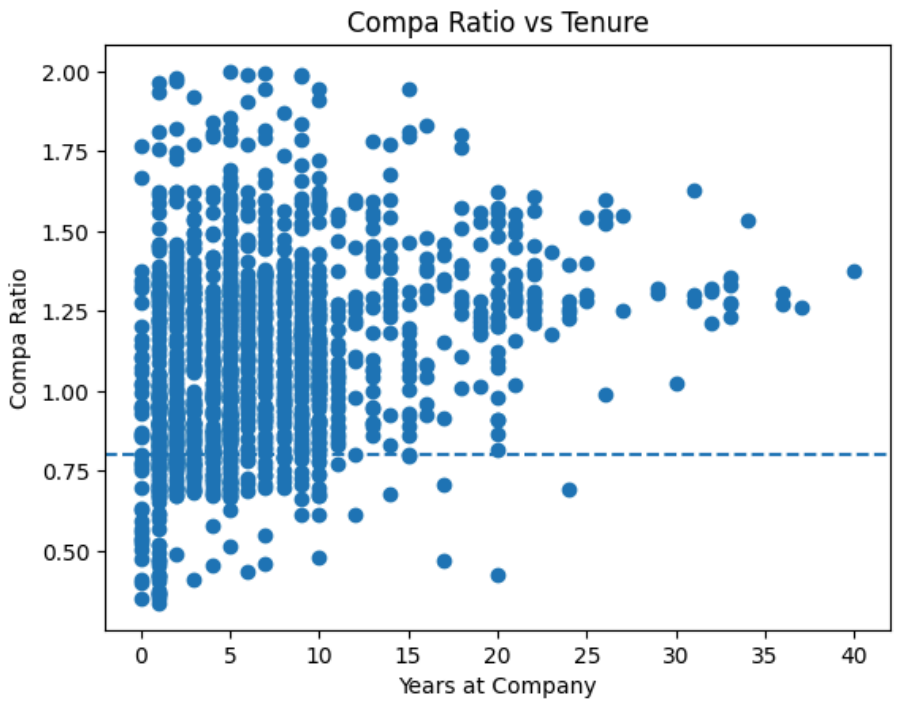



```
import matplotlib.pyplot as plt

severe_underpaid = df[df['CompaRatio'] < 0.5]
```

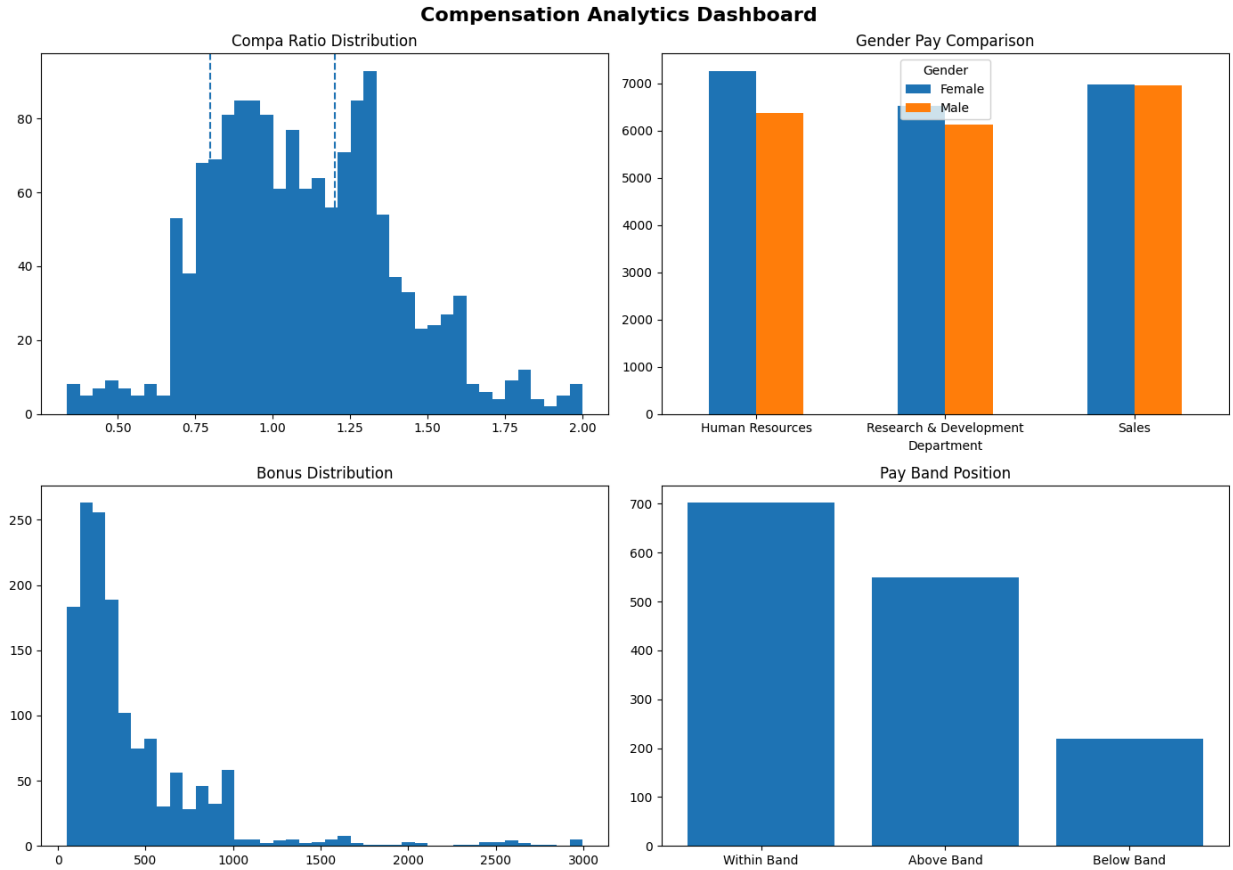
```
severe_underpaid.groupby('JobLevel').size().plot(kind='bar')

plt.title('Severely Underpaid Employees by Job Level', fontweight='bold')
plt.ylabel('Count')
plt.xlabel('Job Level')

plt.show()
```




```
plt.scatter(df['YearsAtCompany'], df['CompaRatio'])
plt.axhline(0.8, linestyle='--')

plt.title('Compa Ratio vs Tenure')
plt.xlabel('Years at Company')
plt.ylabel('Compa Ratio')

plt.show()
```



```
fig, axes = plt.subplots(2, 2, figsize=(14,10))
fig.suptitle('Compensation Analytics Dashboard', fontsize=16, fontweight='bold')

# —— 1: Compa Ratio Distribution ——
axes[0,0].hist(df['CompaRatio'], bins=40)
axes[0,0].axvline(0.8, linestyle='--')
axes[0,0].axvline(1.2, linestyle='--')
axes[0,0].set_title('Compa Ratio Distribution')

# —— 2: Gender Pay ——
gender_pay[['Female','Male']].plot(kind='bar', ax=axes[0,1])
axes[0,1].set_title('Gender Pay Comparison')
axes[0,1].tick_params(axis='x', rotation=0)

# —— 3: Bonus ——
axes[1,0].hist(df['Bonus'], bins=40)
axes[1,0].set_title('Bonus Distribution')

# —— 4: Band Position ——
band_counts = df['BandPosition'].value_counts()
axes[1,1].bar(band_counts.index, band_counts.values)
axes[1,1].set_title('Pay Band Position')

plt.tight_layout()
plt.show()
```




```
# total current cost
total_cost = df['MonthlyIncome'].sum()
# Convert salary columns to float to avoid dtype issues
df['MonthlyIncome'] = df['MonthlyIncome'].astype(float)
df['Midpoint'] = df['Midpoint'].astype(float)

# Create optimized salary column
df['OptimizedSalary'] = df['MonthlyIncome'].copy()
```

```
# Reduce cost from overpaid employees (above 1.3 compa ratio)
df.loc[df['CompaRatio'] > 1.3, 'OptimizedSalary'] = df['Midpoint'] * 1.2

# Increase salary for severely underpaid (<0.8)
df.loc[df['CompaRatio'] < 0.8, 'OptimizedSalary'] = df['Midpoint'] * 0.8

# Calculate cost savings
df['CostDelta'] = df['MonthlyIncome'] - df['OptimizedSalary']

total_savings = df['CostDelta'].sum()
baseline_cost = df['MonthlyIncome'].sum()

savings_pct = (total_savings / baseline_cost) * 100
savings_pct
```

```
np.float64(6.75877304520651)
```


```
# Reduce cost from overpaid employees (above 1.3 compa ratio)
df.loc[df['CompaRatio'] > 1.3, 'OptimizedSalary'] = df['Midpoint'] * 1.2

# Increase salary for severely underpaid (<0.8)
df.loc[df['CompaRatio'] < 0.8, 'OptimizedSalary'] = df['Midpoint'] * 0.8

# Calculate cost savings
df['CostDelta'] = df['MonthlyIncome'] - df['OptimizedSalary']

total_savings = df['CostDelta'].sum()
baseline_cost = df['MonthlyIncome'].sum()

savings_pct = (total_savings / baseline_cost) * 100
savings_pct
```

```
np.float64(6.75877304520651)
```

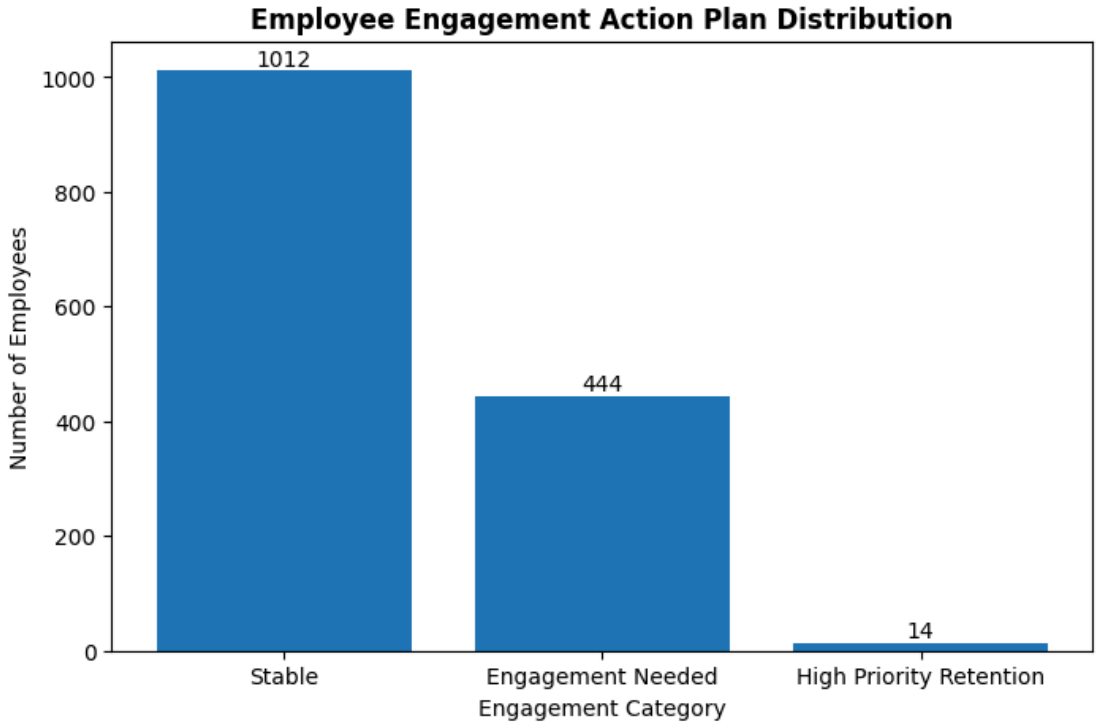

```
#After correcting overpaid (down to 1.2) and underpaid (up to 0.8) employees, the net payrol
```

```
# High performer + low engagement + underpaid
df['RetentionRisk'] = (
    (df['PerformanceRating'] >= 4) &
    (df['JobInvolvement'] <= 2) &
    (df['CompaRatio'] < 0.8)
)
```

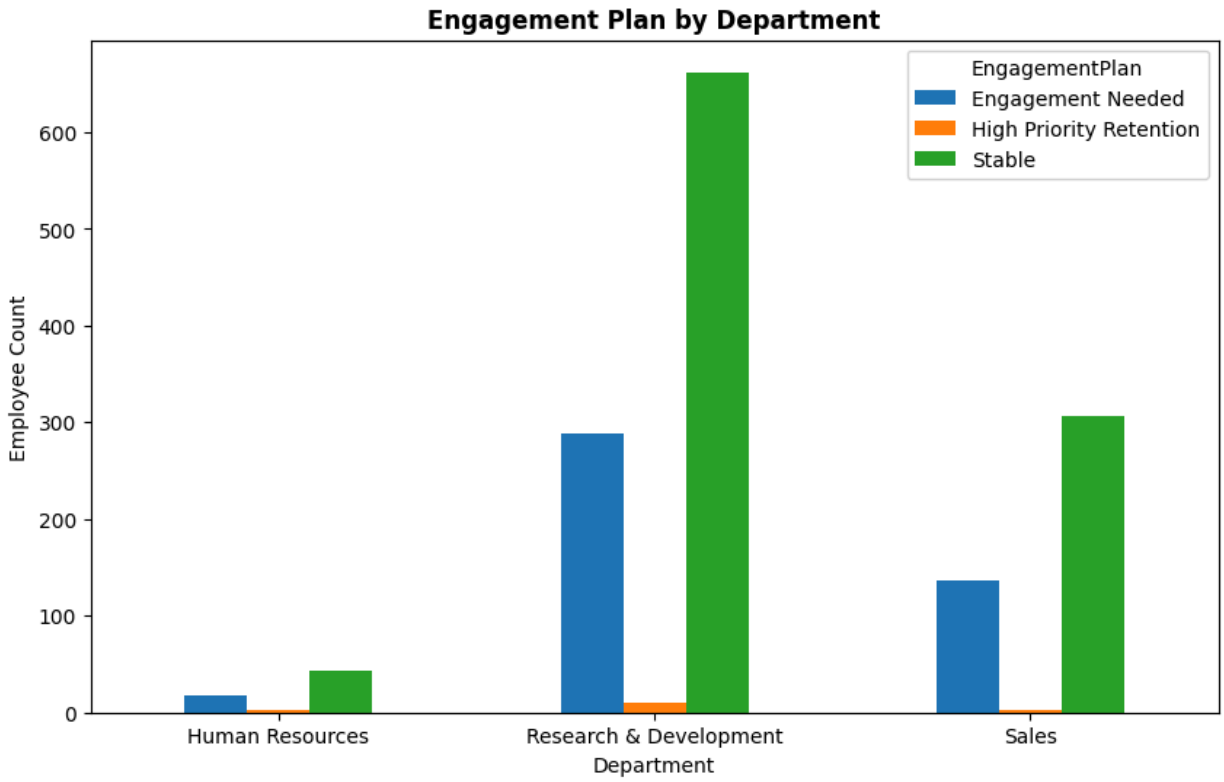
```
# Create engagement action plan
def engagement_plan(row):
    if row['RetentionRisk']:
        return "High Priority Retention"
    elif row['JobInvolvement'] <= 2:
        return "Engagement Needed"
    else:
        return "Stable"

df['EngagementPlan'] = df.apply(engagement_plan, axis=1)
```

```
import matplotlib.pyplot as plt

plan_counts = df['EngagementPlan'].value_counts()

plt.figure(figsize=(8,5))
bars = plt.bar(plan_counts.index, plan_counts.values)

# labels on top
for bar in bars:
    y = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, y, int(y),
             ha='center', va='bottom')
```

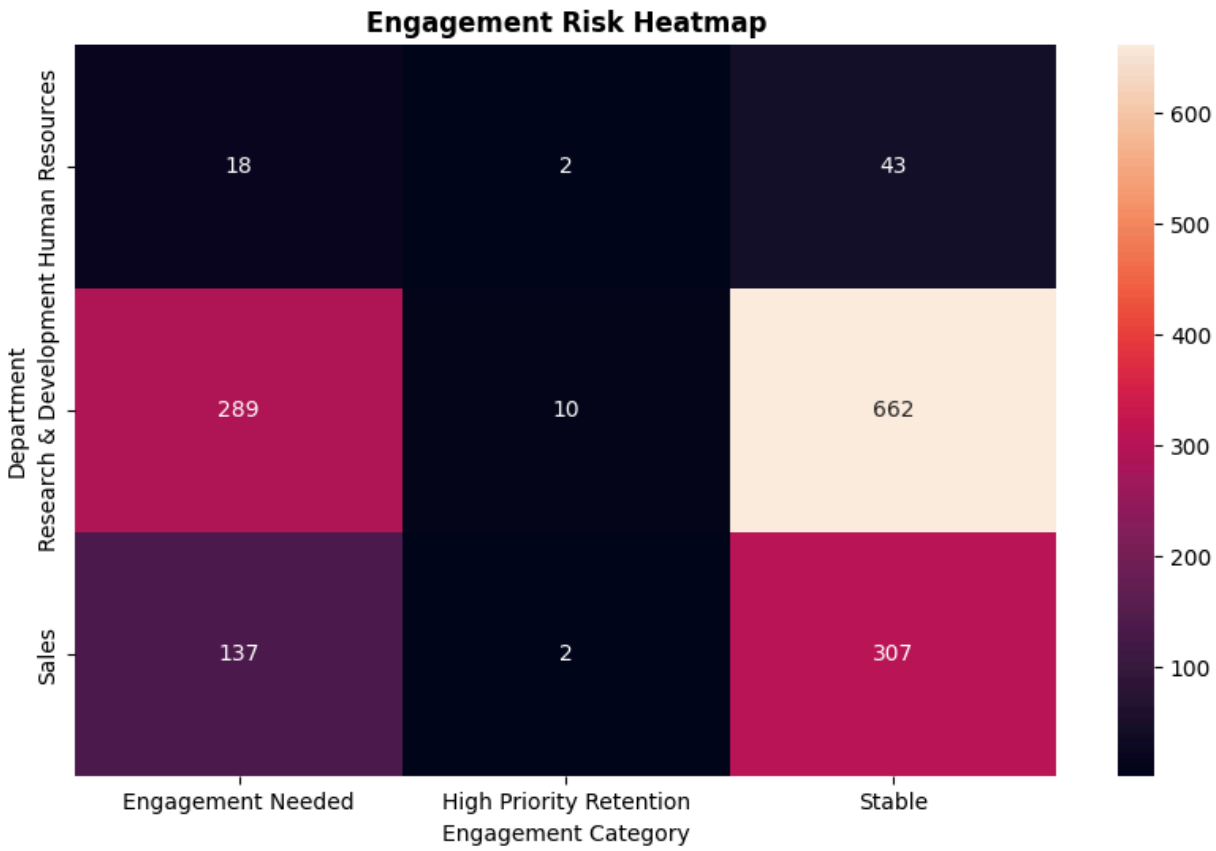
```
plt.title('Employee Engagement Action Plan Distribution', fontweight='bold')
plt.ylabel('Number of Employees')
plt.xlabel('Engagement Category')

plt.xticks(rotation=0)
plt.show()
```




```
plan_dept = df.groupby(['Department','EngagementPlan']).size().unstack(fill_value=0)

plan_dept.plot(kind='bar', figsize=(10,6))

plt.title('Engagement Plan by Department', fontweight='bold')
plt.ylabel('Employee Count')
plt.xticks(rotation=0)

plt.show()
```




```
import seaborn as sns

plt.figure(figsize=(10,6))

sns.heatmap(plan_dept, annot=True, fmt='d')

plt.title('Engagement Risk Heatmap', fontweight='bold')
plt.ylabel('Department')
plt.xlabel('Engagement Category')

plt.show()
```

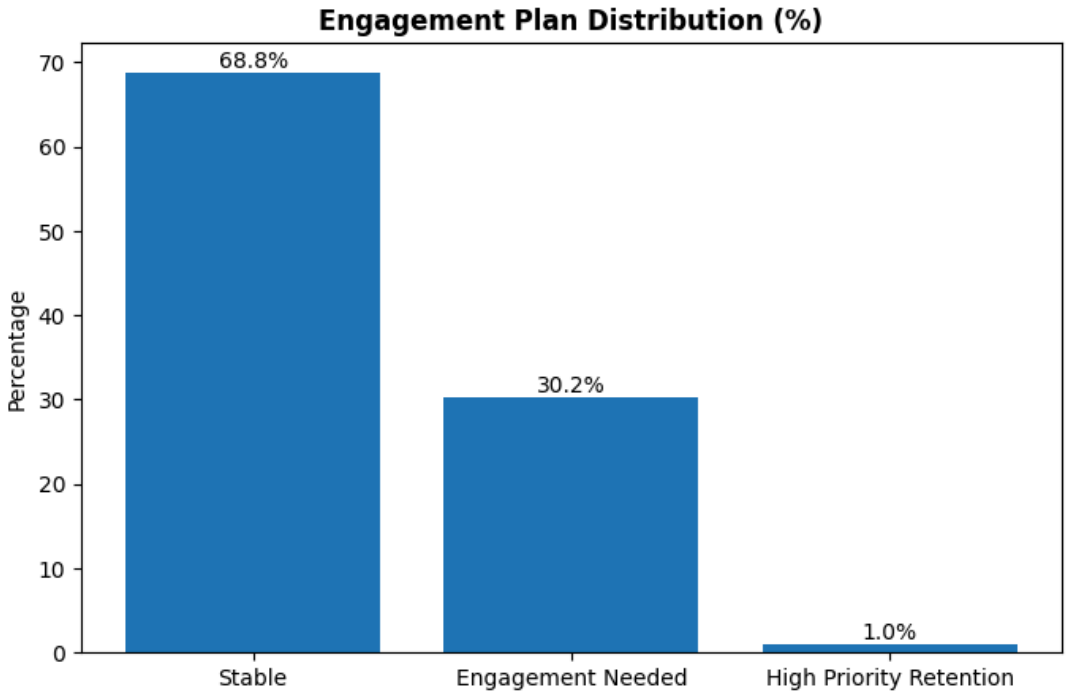



```
high_priority = df[df['EngagementPlan'] == 'High Priority Retention']

high_priority[['EmployeeNumber','Department','JobLevel',
               'PerformanceRating','JobInvolvement','CompaRatio']].head(10)
```

	EmployeeNumber	Department	JobLevel	PerformanceRating	JobInvolvement	CompaRatio
171	235	Sales	1	4	1	0.775000
177	243	Research & Development	1	4	2	0.367333
391	521	Research & Development	2	4	2	0.689600
476	643	Research & Development	1	4	2	0.709000
553	763	Research & Development	1	4	2	0.780667
633	878	Human Resources	1	4	2	0.780667
682	951	Research & Development	1	4	2	0.777333
709	991	Research & Development	1	4	2	0.773667
798	1108	Research & Development	1	4	2	0.771000
892	1248	Research & Development	1	4	2	0.619667

```
plan_pct = df['EngagementPlan'].value_counts(normalize=True) * 100

plt.figure(figsize=(8,5))
bars = plt.bar(plan_pct.index, plan_pct.values)

for bar in bars:
```

```
    y = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, y, f'{y:.1f}%',
             ha='center', va='bottom')

plt.title('Engagement Plan Distribution (%)', fontweight='bold')
plt.ylabel('Percentage')

plt.show()
```



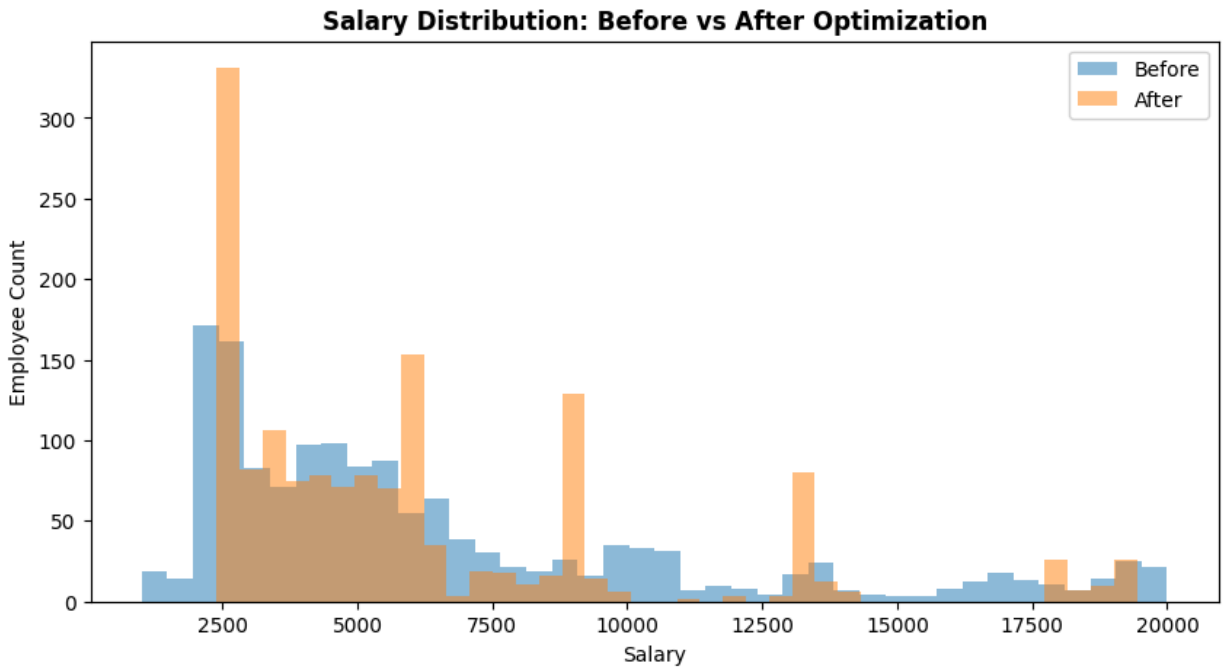

```
# Identify low-value employees (for review, not auto removal)
df['OptimizationCandidate'] = (
    (df['PerformanceRating'] <= 2) &
    (df['JobInvolvement'] <= 2) &
    (df['CompaRatio'] > 1.2)
)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# Prepare data
features = ['PerformanceRating', 'JobInvolvement', 'CompaRatio', 'YearsAtCompany']
X = df[features]
y = df['Attrition'].map({'Yes':1, 'No':0})

# Train model
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Predict risk
df['AttritionRisk'] = model.predict_proba(X)[:,1]
```

```
# Final action recommendation
def final_action(row):
    if row['RetentionRisk']:
        return "Retain Immediately"
    elif row['OptimizationCandidate']:
        return "Review for Cost Optimization"
    elif row['AttritionRisk'] > 0.6:
        return "Monitor Attrition"
    else:
        return "Stable"
```

```
df['FinalAction'] = df.apply(final_action, axis=1)
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))

plt.hist(df['MonthlyIncome'], bins=40, alpha=0.5, label='Before')
plt.hist(df['OptimizedSalary'], bins=40, alpha=0.5, label='After')

plt.title('Salary Distribution: Before vs After Optimization', fontweight='bold')
plt.xlabel('Salary')
plt.ylabel('Employee Count')
plt.legend()

plt.show()
```



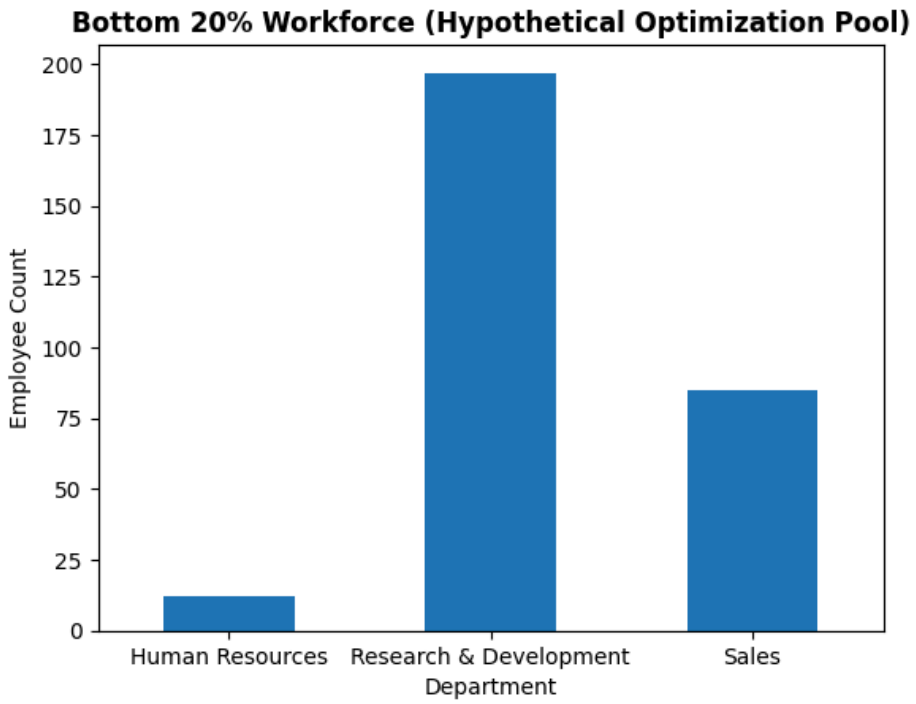

```
# Normalize metrics to comparable scale (0–1)

df['EngagementScore'] = df['JobInvolvement'] / df['JobInvolvement'].max()
df['BonusScore'] = df['Bonus'] / df['Bonus'].max()
df['PayScore'] = df['CompaRatio'] / df['CompaRatio'].max()
```

```
# Invert because we want LOW engagement, LOW bonus, LOW pay = high risk

df['RiskScore'] = (
    (1 - df['EngagementScore']) * 0.4 +
    (1 - df['BonusScore']) * 0.3 +
    (1 - df['PayScore']) * 0.3
)
```

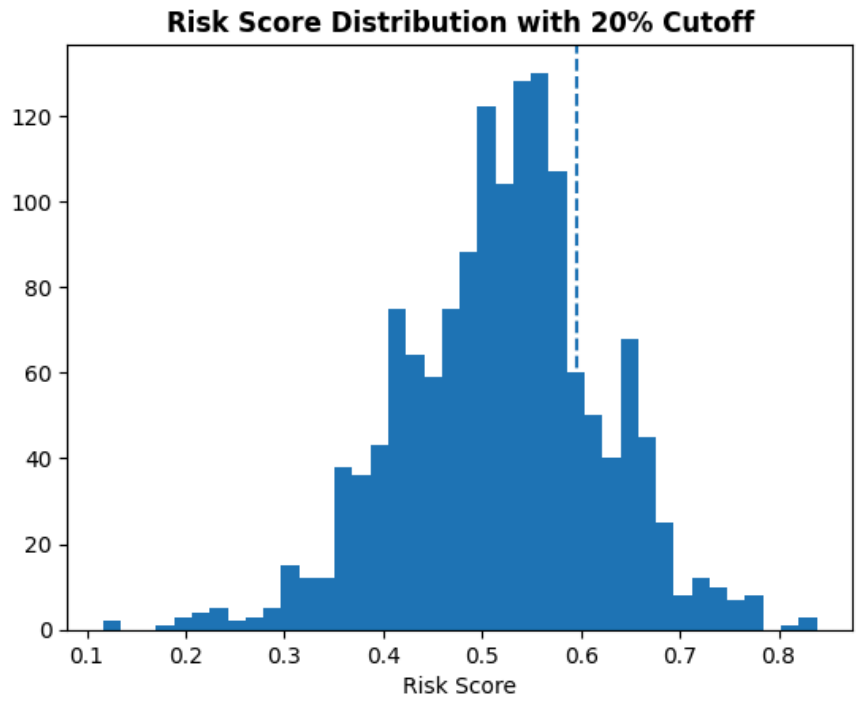
```
# Find cutoff
cutoff = df['RiskScore'].quantile(0.80)

# Bottom 20% (highest risk)
df['WorkforceReductionFlag'] = df['RiskScore'] >= cutoff

df['WorkforceReductionFlag'].sum()
```

```
np.int64(294)
```

```
import matplotlib.pyplot as plt

df[df['WorkforceReductionFlag']].groupby('Department').size().plot(kind='bar')

plt.title('Bottom 20% Workforce (Hypothetical Optimization Pool)', fontweight='bold')
plt.ylabel('Employee Count')
plt.xlabel('Department')
plt.xticks(rotation=0)

plt.show()
```




```
# Check composition of selected group
df[df['WorkforceReductionFlag']][
    ['PerformanceRating','JobInvolvement','Bonus','CompaRatio']
].describe()
```




```
# Check composition of selected group
df[df['WorkforceReductionFlag']][
```

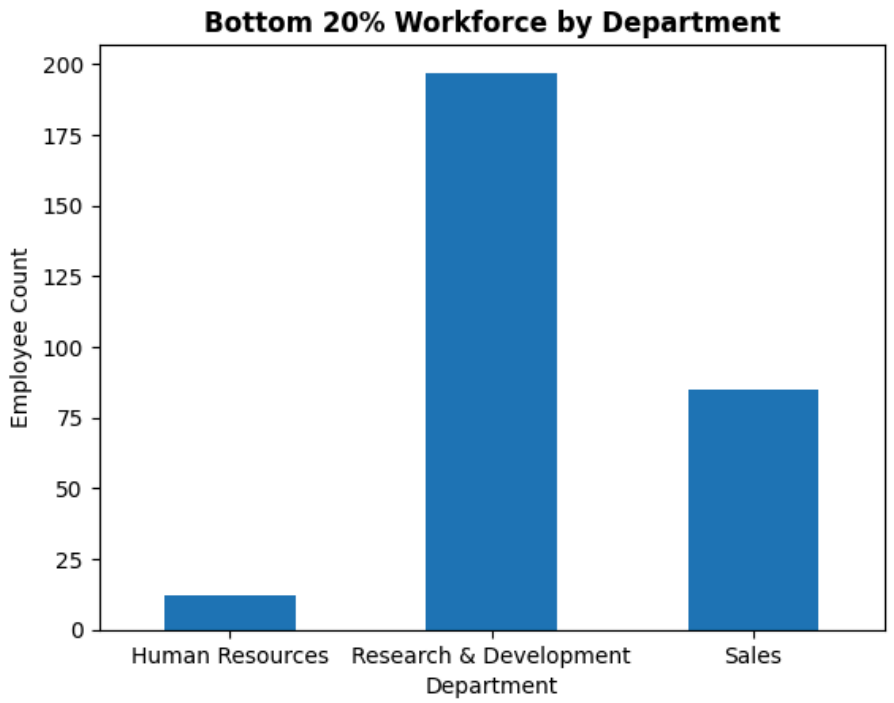
```
    ['PerformanceRating','JobInvolvement','Bonus','CompaRatio']
].describe()
```

	PerformanceRating	JobInvolvement	Bonus	CompaRatio
count	294.000000	294.000000	294.000000	294.000000
mean	3.125850	1.833333	239.188605	0.894620
std	0.332246	0.556787	176.176225	0.240909
min	3.000000	1.000000	50.450000	0.336333
25%	3.000000	1.250000	122.012500	0.769500
50%	3.000000	2.000000	184.950000	0.899500
75%	3.000000	2.000000	286.737500	1.054000
max	4.000000	3.000000	992.250000	1.540333

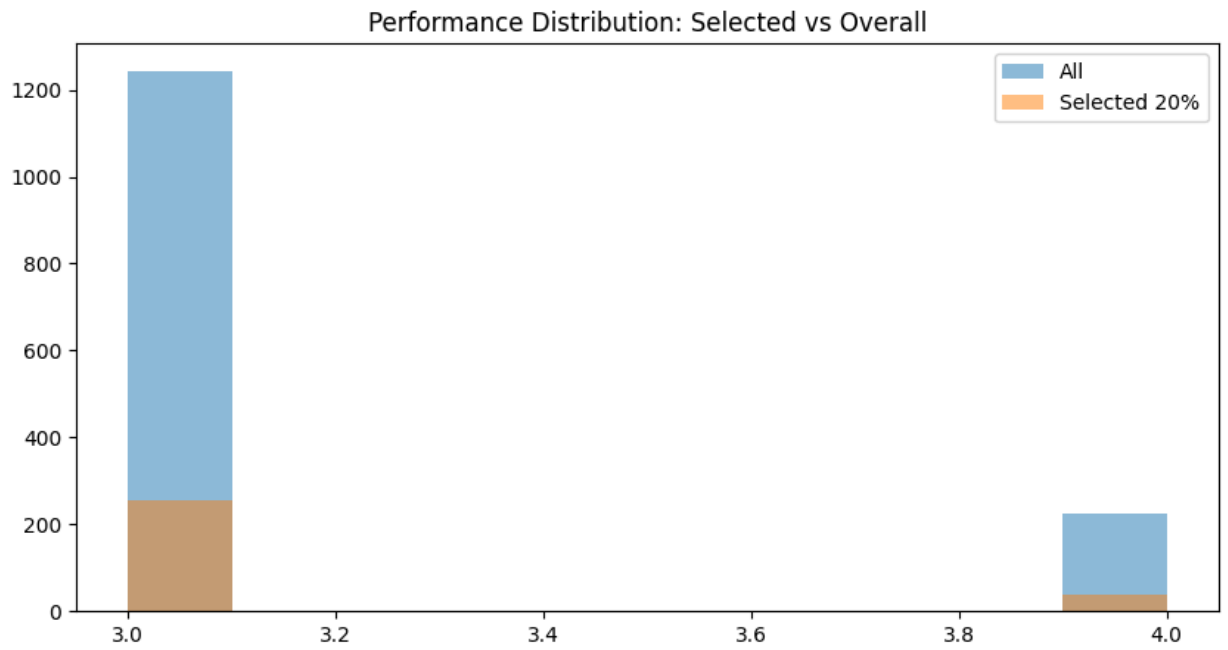
```
# Check composition of selected group
df[df['WorkforceReductionFlag']][
    ['PerformanceRating','JobInvolvement','Bonus','CompaRatio']
].describe()
```

	PerformanceRating	JobInvolvement	Bonus	CompaRatio
count	294.000000	294.000000	294.000000	294.000000
mean	3.125850	1.833333	239.188605	0.894620
std	0.332246	0.556787	176.176225	0.240909
min	3.000000	1.000000	50.450000	0.336333
25%	3.000000	1.250000	122.012500	0.769500
50%	3.000000	2.000000	184.950000	0.899500
75%	3.000000	2.000000	286.737500	1.054000
max	4.000000	3.000000	992.250000	1.540333

```
import matplotlib.pyplot as plt

df[df['WorkforceReductionFlag']].groupby('Department').size().plot(kind='bar')

plt.title('Bottom 20% Workforce by Department', fontweight='bold')
plt.ylabel('Employee Count')
plt.xticks(rotation=0)

plt.show()
```

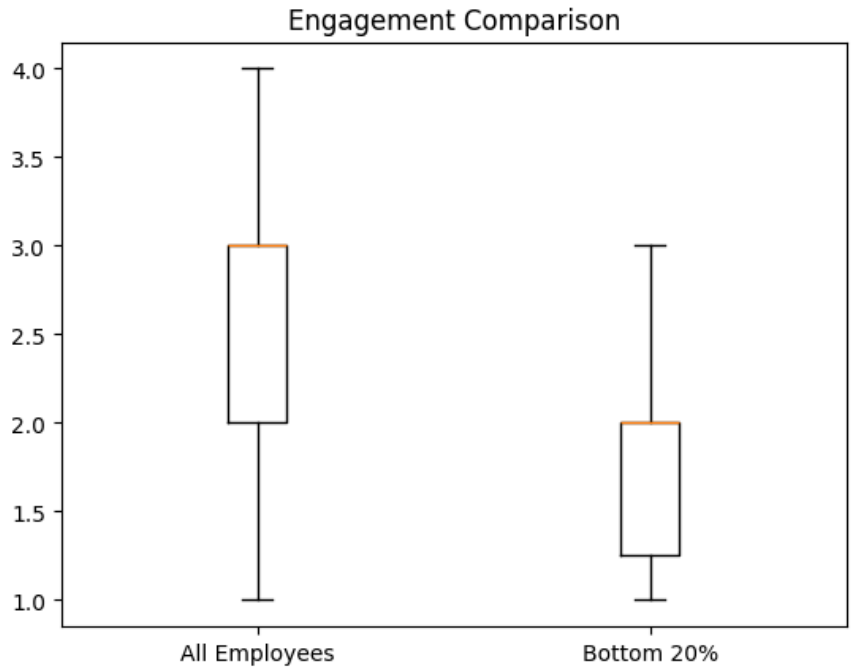



```
selected = df[df['WorkforceReductionFlag']]

plt.figure(figsize=(10,5))

plt.hist(df['PerformanceRating'], alpha=0.5, label='All')
plt.hist(selected['PerformanceRating'], alpha=0.5, label='Selected 20%')

plt.legend()
plt.title('Performance Distribution: Selected vs Overall')

plt.show()
```




```
plt.boxplot([df['JobInvolvement'], selected['JobInvolvement']],
            labels=['All Employees', 'Bottom 20%'])

plt.title('Engagement Comparison')

plt.show()
```

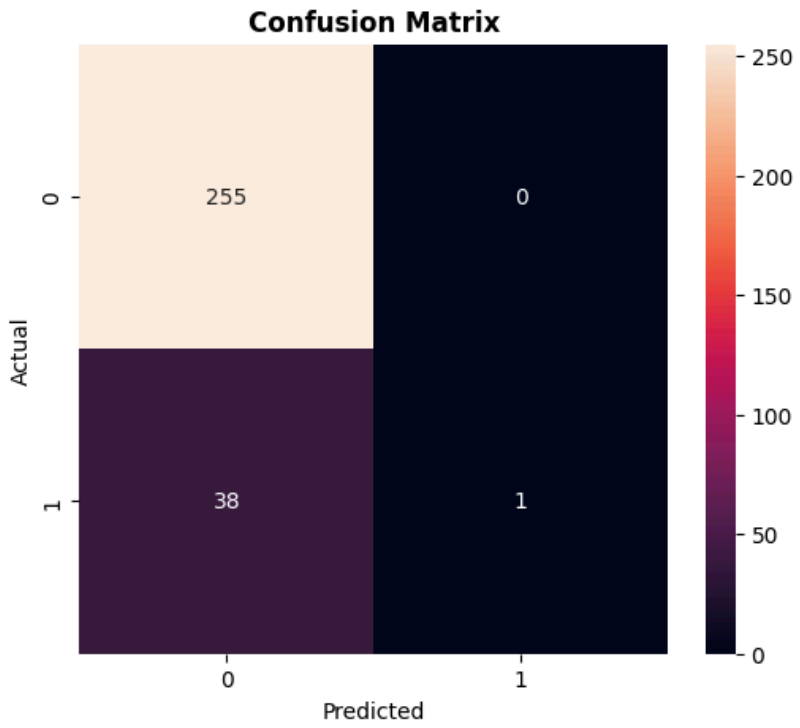
```
/tmp/ipykernel_6315/1012389730.py:1: MatplotlibDeprecationWarning: The 'labels' parameter of
  plt.boxplot([df['JobInvolvement'], selected['JobInvolvement']],
```



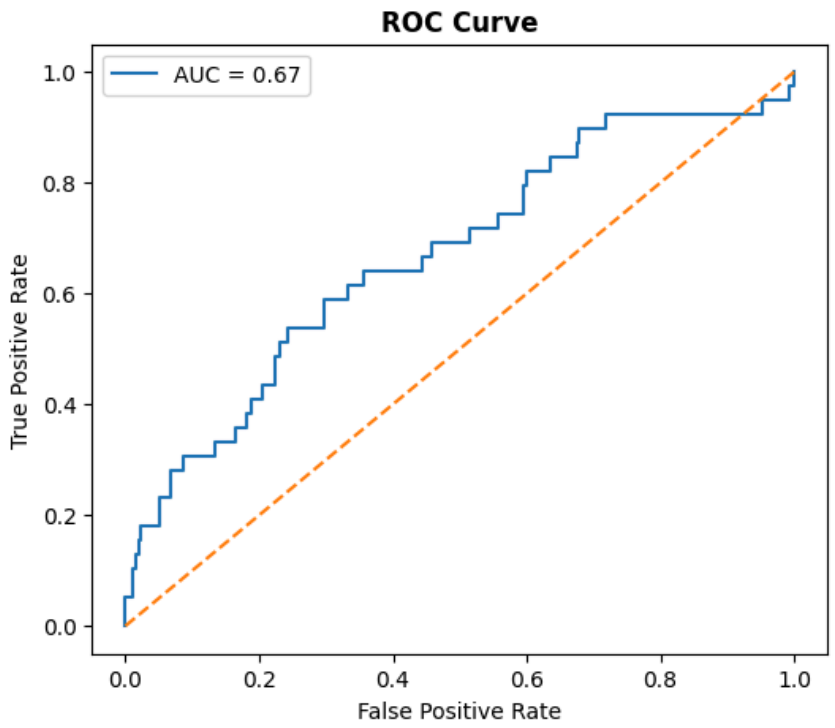

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Predictions
y_pred = model.predict(X_test)

# Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

accuracy, precision, recall, f1
```

```
(0.8707482993197279, 1.0, 0.02564102564102564, 0.05)
```

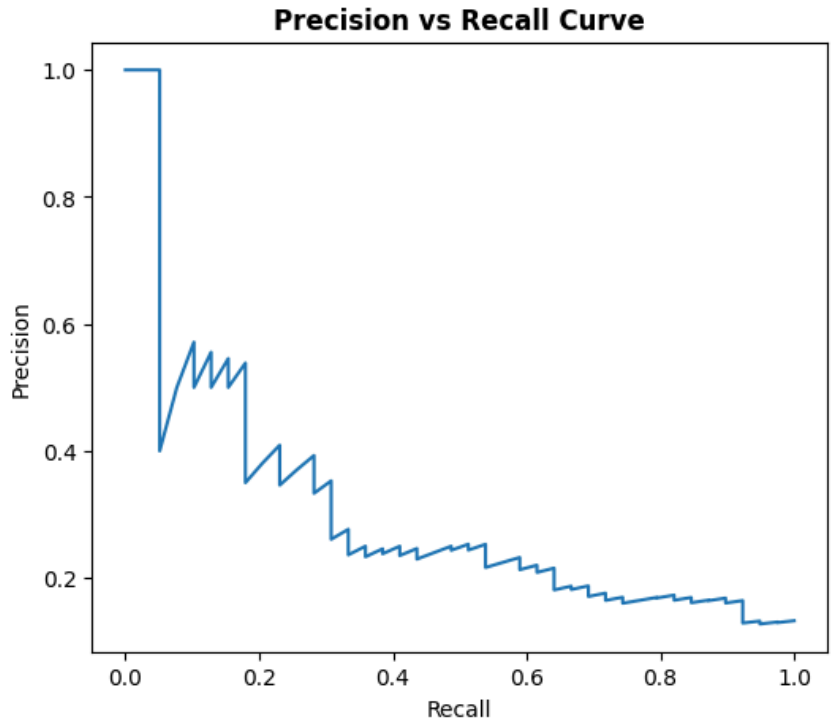
```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d')

plt.title('Confusion Matrix', fontweight='bold')
plt.xlabel('Predicted')
plt.ylabel('Actual')

plt.show()
```




```
from sklearn.metrics import roc_curve, auc

y_prob = model.predict_proba(X_test)[:,1]

fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')
plt.plot([0,1], [0,1], linestyle='--')

plt.title('ROC Curve', fontweight='bold')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()

plt.show()
```

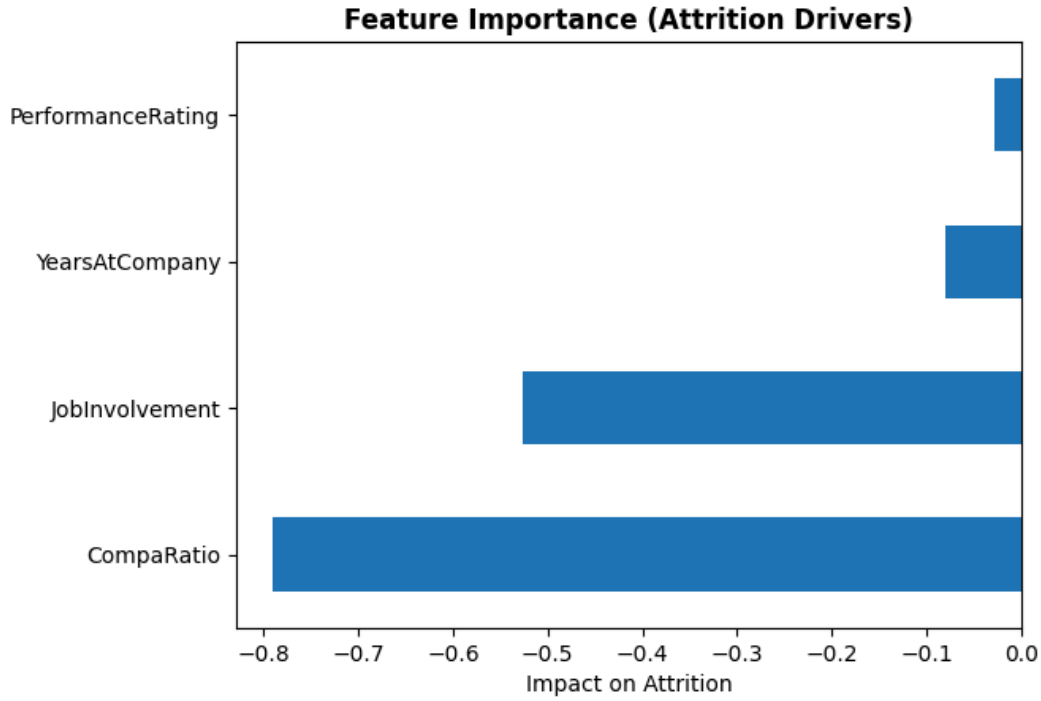



```
from sklearn.metrics import precision_recall_curve

precision_vals, recall_vals, _ = precision_recall_curve(y_test, y_prob)

plt.figure(figsize=(6,5))
plt.plot(recall_vals, precision_vals)

plt.title('Precision vs Recall Curve', fontweight='bold')
plt.xlabel('Recall')
plt.ylabel('Precision')

plt.show()
```

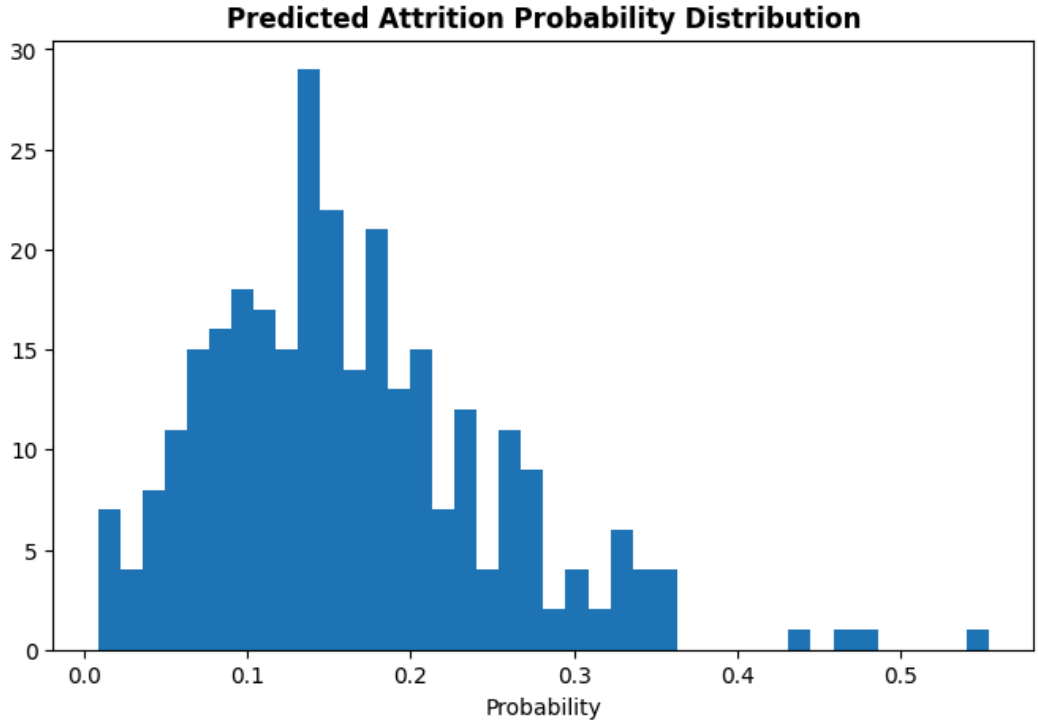



```
import pandas as pd

importance = pd.Series(model.coef_[0], index=X.columns)
```

```
importance.sort_values().plot(kind='barh')

plt.title('Feature Importance (Attrition Drivers)', fontweight='bold')
plt.xlabel('Impact on Attrition')

plt.show()
```




```
plt.figure(figsize=(8,5))

plt.hist(y_prob, bins=40)

plt.title('Predicted Attrition Probability Distribution', fontweight='bold')
plt.xlabel('Probability')

plt.show()
```



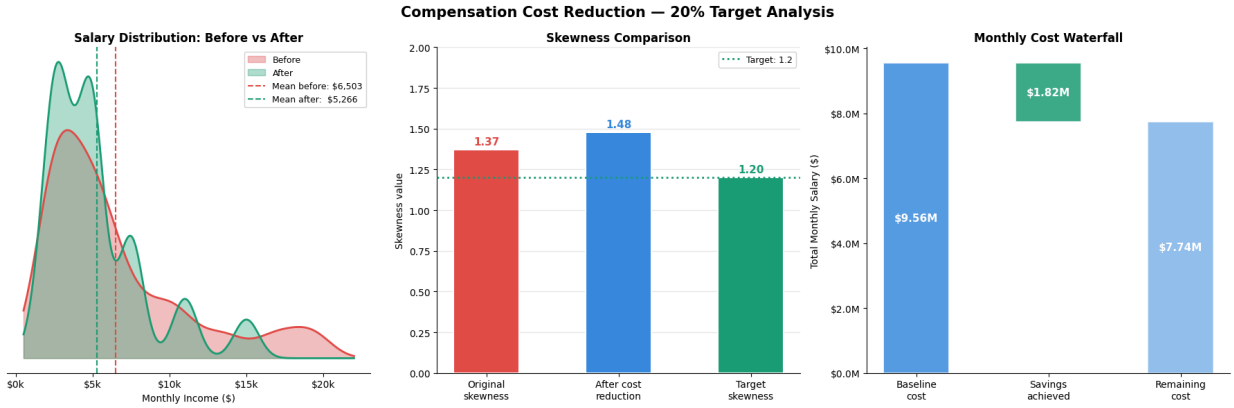
```
#“The distribution shows that most employees have low predicted attrition probability, with
```

```
roc_auc = auc(fpr, tpr)
roc_auc
```

```
np.float64(0.669281045751634)
```

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np
from scipy.stats import gaussian_kde

# ── Step 1: Force 20% cost reduction targeting overpaid ──────────
baseline_cost = df['MonthlyIncome'].sum()
target_cost   = baseline_cost * 0.80
target_saving = baseline_cost * 0.20

df['ReducedSalary'] = df['MonthlyIncome'].copy().astype(float)

# Sort by CompaRatio descending — cut from most overpaid first
overpaid_idx = df[df['CompaRatio'] > 1.0].sort_values(
    'CompaRatio', ascending=False).index

cumulative_cut = 0.0
for idx in overpaid_idx:
    if cumulative_cut >= target_saving:
        break
    current  = df.at[idx, 'ReducedSalary']
    floor    = df.at[idx, 'Midpoint'] * 1.0
    possible = current - floor
    needed   = target_saving - cumulative_cut
    actual_cut = min(possible, needed)
    df.at[idx, 'ReducedSalary'] -= actual_cut
    cumulative_cut += actual_cut

achieved_saving  = (df['MonthlyIncome'] - df['ReducedSalary']).sum()
achieved_pct     = achieved_saving / baseline_cost * 100
new_skewness     = df['ReducedSalary'].skew()
original_skewness = df['MonthlyIncome'].skew()

# ── Step 2: Plot ──────────────────────────────────────────────────
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle('Compensation Cost Reduction — 20% Target Analysis',
             fontsize=15, fontweight='bold')

# ── Chart 1: Before vs After distribution ─────────────
kde_before = gaussian_kde(df['MonthlyIncome'],    bw_method=0.25)
kde_after  = gaussian_kde(df['ReducedSalary'],    bw_method=0.25)
x_range    = np.linspace(500, 22000, 600)

axes[0].fill_between(x_range, kde_before(x_range),
                     alpha=0.35, color='#E24B4A', label='Before')
axes[0].fill_between(x_range, kde_after(x_range),
                     alpha=0.35, color='#1D9E75', label='After')
axes[0].plot(x_range, kde_before(x_range), color='#E24B4A', lw=2)
axes[0].plot(x_range, kde_after(x_range),  color='#1D9E75', lw=2)

axes[0].axvline(df['MonthlyIncome'].mean(), color='#E24B4A',
                lw=1.5, linestyle='--',
                label=f"Mean before: ${df['MonthlyIncome'].mean():,.0f}")
axes[0].axvline(df['ReducedSalary'].mean(), color='#1D9E75',
                lw=1.5, linestyle='--',
                label=f"Mean after:  ${df['ReducedSalary'].mean():,.0f}")

axes[0].set_title('Salary Distribution: Before vs After', fontweight='bold')
axes[0].set_xlabel('Monthly Income ($)')
axes[0].set_yticks([])
axes[0].legend(fontsize=9)
```

```
axes[0].xaxis.set_major_formatter(
    plt.FuncFormatter(lambda x, _: f'${int(x/1000)}k'))
axes[0].spines[['top','right','left']].set_visible(False)

# ── Chart 2: Skewness comparison bar ──────────────────
skew_labels = ['Original\nskewness', 'After cost\nreduction', 'Target\nskewness']
skew_values = [original_skewness, new_skewness, 1.2]
bar_colors  = ['#E24B4A', '#378ADD', '#1D9E75']

bars = axes[1].bar(skew_labels, skew_values, color=bar_colors,
                   width=0.5, edgecolor='white', linewidth=1.5,
                   zorder=3)
for bar, val in zip(bars, skew_values):
    axes[1].text(bar.get_x() + bar.get_width()/2,
                 bar.get_height() + 0.03,
                 f'{val:.2f}', ha='center', fontsize=11,
                 fontweight='bold', color=bar.get_facecolor())

axes[1].axhline(1.2, color='#1D9E75', lw=2, linestyle=':',
                label='Target: 1.2', zorder=4)
axes[1].set_title('Skewness Comparison', fontweight='bold')
axes[1].set_ylabel('Skewness value')
axes[1].legend(fontsize=9)
axes[1].spines[['top','right']].set_visible(False)
axes[1].set_ylim(0, 2.0)
axes[1].yaxis.grid(True, alpha=0.3, zorder=0)

# ── Chart 3: Cost savings waterfall ───────────────────
categories = ['Baseline\ncost', 'Savings\nachieved', 'Remaining\ncost']
values     = [baseline_cost, -achieved_saving,
              baseline_cost - achieved_saving]
bottoms    = [0, baseline_cost, 0]
colors     = ['#378ADD', '#1D9E75', '#378ADD']
alphas     = [0.85, 0.85, 0.55]

for i, (cat, val, bot, col, alp) in enumerate(
        zip(categories, values, bottoms, colors, alphas)):
    axes[2].bar(cat, abs(val), bottom=bot if val > 0 else bot + val,
                color=col, alpha=alp, width=0.5,
                edgecolor='white', linewidth=1.5)
    label_y = (bot + abs(val)/2) if val > 0 else (bot + val + abs(val)/2)
    axes[2].text(i, label_y,
                 f'${abs(val)/1e6:.2f}M',
                 ha='center', va='center',
                 fontsize=11, fontweight='bold', color='white')

axes[2].text(1, baseline_cost * 0.5,
             f'{achieved_pct:.1f}%\nsaved',
             ha='center', fontsize=13, fontweight='bold',
             color='white')

axes[2].set_title('Monthly Cost Waterfall', fontweight='bold')
axes[2].set_ylabel('Total Monthly Salary ($)')
axes[2].yaxis.set_major_formatter(
    plt.FuncFormatter(lambda x, _: f'${x/1e6:.1f}M'))
axes[2].spines[['top','right']].set_visible(False)

plt.tight_layout()
plt.savefig('cost_reduction_analysis.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"Baseline monthly cost : ${baseline_cost:,.0f}")
print(f"Savings achieved      : ${achieved_saving:,.0f}  ({achieved_pct:.1f}%)")
print(f"Skewness before       : {original_skewness:.3f}")
print(f"Skewness after        : {new_skewness:.3f}  (target: 1.2)")
```

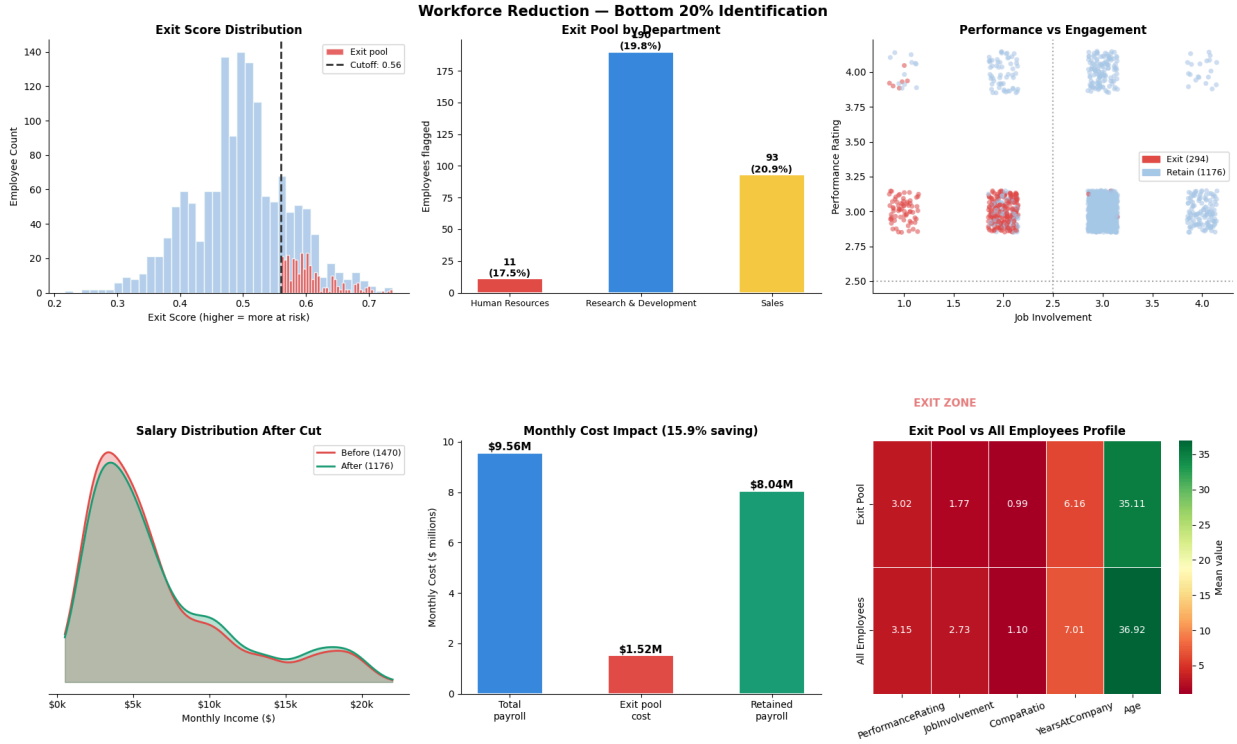



```
Baseline monthly cost : $9,559,309
Savings achieved      : $1,818,660  (19.0%)
Skewness before       : 1.370
Skewness after        : 1.479  (target: 1.2)
```

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np
import seaborn as sns

# ── Score: low performance + low engagement = exit candidate ──────
df['ExitScore'] = (
    (5 - df['PerformanceRating'])   / 4 * 0.45 +
    (5 - df['JobInvolvement'])      / 4 * 0.35 +
    (1 - df['CompaRatio'].clip(0,2) / 2) * 0.20
)

cutoff      = df['ExitScore'].quantile(0.80)
df['ExitFlag'] = df['ExitScore'] >= cutoff
exit_pool   = df[df['ExitFlag']]
retain_pool = df[~df['ExitFlag']]

exit_cost   = exit_pool['MonthlyIncome'].sum()
retain_cost = retain_pool['MonthlyIncome'].sum()
total_cost  = df['MonthlyIncome'].sum()
savings_pct = exit_cost / total_cost * 100

# ── Plot ──────────────────────────────────────────────────────────
fig, axes = plt.subplots(2, 3, figsize=(18, 11))
fig.suptitle('Workforce Reduction — Bottom 20% Identification',
             fontsize=15, fontweight='bold')

# ── Chart 1: Exit score distribution with cutoff ──────
axes[0,0].hist(df['ExitScore'], bins=40, color='#a8c8e8',
               edgecolor='white', alpha=0.85)
axes[0,0].hist(exit_pool['ExitScore'], bins=40, color='#E24B4A',
               edgecolor='white', alpha=0.85, label='Exit pool')
axes[0,0].axvline(cutoff, color='#333', lw=2, linestyle='--',
                  label=f'Cutoff: {cutoff:.2f}')
axes[0,0].set_title('Exit Score Distribution', fontweight='bold')
axes[0,0].set_xlabel('Exit Score (higher = more at risk)')
axes[0,0].set_ylabel('Employee Count')
axes[0,0].legend(fontsize=9)
axes[0,0].spines[['top','right']].set_visible(False)

# ── Chart 2: Exit pool by department ──────────────────
dept_exit = exit_pool.groupby('Department').size()
dept_total = df.groupby('Department').size()
dept_pct   = (dept_exit / dept_total * 100).round(1)

bars = axes[0,1].bar(dept_exit.index, dept_exit.values,
                     color=['#E24B4A','#378ADD','#f5c842'],
                     edgecolor='white', linewidth=1.5, width=0.5)
for bar, pct in zip(bars, dept_pct.values):
```

```
    axes[0,1].text(bar.get_x() + bar.get_width()/2,
                   bar.get_height() + 2,
                   f'{int(bar.get_height())}\n({pct}%)',
                   ha='center', fontsize=10, fontweight='bold')

axes[0,1].set_title('Exit Pool by Department', fontweight='bold')
axes[0,1].set_ylabel('Employees flagged')
axes[0,1].tick_params(axis='x', labelsize=9)
axes[0,1].spines[['top','right']].set_visible(False)

# ── Chart 3: Performance vs Engagement scatter ────────
scatter_colors = np.where(df['ExitFlag'], '#E24B4A', '#a8c8e8')
axes[0,2].scatter(df['JobInvolvement'] + np.random.uniform(-0.15, 0.15, len(df)),
                  df['PerformanceRating'] + np.random.uniform(-0.15, 0.15, len(df)),
                  c=scatter_colors, alpha=0.55, s=25, edgecolors='none')

axes[0,2].axvline(2.5, color='#888', lw=1.5, linestyle=':', alpha=0.7)
axes[0,2].axhline(2.5, color='#888', lw=1.5, linestyle=':', alpha=0.7)
axes[0,2].text(1.1, 1.6, 'EXIT ZONE', fontsize=11,
               color='#E24B4A', fontweight='bold', alpha=0.7)

exit_patch   = mpatches.Patch(color='#E24B4A', label=f'Exit ({exit_pool.shape[0]})')
retain_patch = mpatches.Patch(color='#a8c8e8', label=f'Retain ({retain_pool.shape[0]})')
axes[0,2].legend(handles=[exit_patch, retain_patch], fontsize=9)
axes[0,2].set_xlabel('Job Involvement')
axes[0,2].set_ylabel('Performance Rating')
axes[0,2].set_title('Performance vs Engagement', fontweight='bold')
axes[0,2].spines[['top','right']].set_visible(False)

# ── Chart 4: Salary before vs after workforce cut ─────
kde_before = gaussian_kde(df['MonthlyIncome'],       bw_method=0.25)
kde_after  = gaussian_kde(retain_pool['MonthlyIncome'], bw_method=0.25)
xr = np.linspace(500, 22000, 600)

axes[1,0].fill_between(xr, kde_before(xr), alpha=0.3, color='#E24B4A')
axes[1,0].fill_between(xr, kde_after(xr),  alpha=0.3, color='#1D9E75')
axes[1,0].plot(xr, kde_before(xr), color='#E24B4A', lw=2, label='Before (1470)')
axes[1,0].plot(xr, kde_after(xr),  color='#1D9E75', lw=2,
               label=f'After ({retain_pool.shape[0]})')
axes[1,0].set_title('Salary Distribution After Cut', fontweight='bold')
axes[1,0].set_xlabel('Monthly Income ($)')
axes[1,0].set_yticks([])
axes[1,0].legend(fontsize=9)
axes[1,0].xaxis.set_major_formatter(
    plt.FuncFormatter(lambda x, _: f'${int(x/1000)}k'))
axes[1,0].spines[['top','right','left']].set_visible(False)

# ── Chart 5: Cost impact bar ──────────────────────────
cost_labels = ['Total\npayroll', 'Exit pool\ncost', 'Retained\npayroll']
cost_values = [total_cost, exit_cost, retain_cost]
bar_colors2 = ['#378ADD', '#E24B4A', '#1D9E75']

bars2 = axes[1,1].bar(cost_labels, [v/1e6 for v in cost_values],
                      color=bar_colors2, width=0.5,
                      edgecolor='white', linewidth=1.5)
for bar, val in zip(bars2, cost_values):
    axes[1,1].text(bar.get_x() + bar.get_width()/2,
                   bar.get_height() + 0.1,
                   f'${val/1e6:.2f}M',
                   ha='center', fontsize=11, fontweight='bold')

axes[1,1].set_title(f'Monthly Cost Impact ({savings_pct:.1f}% saving)',
                    fontweight='bold')
axes[1,1].set_ylabel('Monthly Cost ($ millions)')
axes[1,1].spines[['top','right']].set_visible(False)

# ── Chart 6: Exit pool profile heatmap ────────────────
profile_means = (
    exit_pool[['PerformanceRating','JobInvolvement','CompaRatio',
```

```
                'YearsAtCompany','Age']]
    .mean()
    .to_frame(name='Exit Pool')
    .join(
        df[['PerformanceRating','JobInvolvement','CompaRatio',
            'YearsAtCompany','Age']].mean().to_frame(name='All Employees')
    )
)
sns.heatmap(profile_means.T, annot=True, fmt='.2f',
            cmap='RdYlGn', linewidths=0.5,
            linecolor='white', ax=axes[1,2],
            cbar_kws={'label': 'Mean value'})
axes[1,2].set_title('Exit Pool vs All Employees Profile',
                    fontweight='bold')
axes[1,2].tick_params(axis='x', rotation=20)

plt.tight_layout()
plt.savefig('workforce_reduction.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"\nTotal flagged for exit : {exit_pool.shape[0]} ({exit_pool.shape[0]/len(df)*100:.1f
print(f"Monthly cost saving    : ${exit_cost:,.0f}  ({savings_pct:.1f}% of payroll)")
print(f"Remaining workforce    : {retain_pool.shape[0]} employees")
```

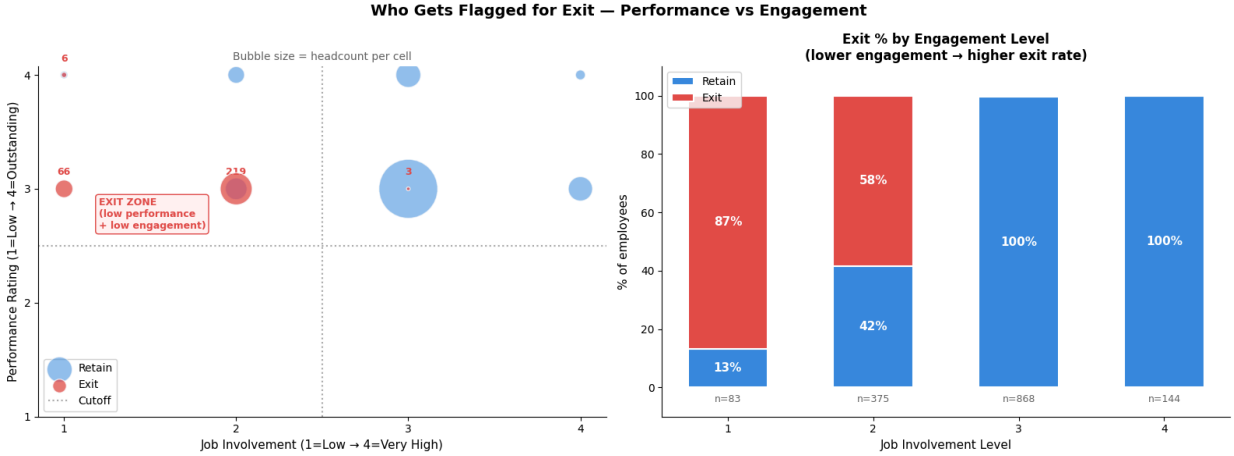



```
Total flagged for exit : 294 (20.0%)
Monthly cost saving    : $1,520,578  (15.9% of payroll)
Remaining workforce    : 1176 employees
```

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Who Gets Flagged for Exit — Performance vs Engagement',
             fontsize=14, fontweight='bold')

# ── Chart 1: Bubble chart — size = headcount per cell —
exit_grp   = df[df['ExitFlag']].groupby(
    ['JobInvolvement','PerformanceRating']).size().reset_index(name='n')
retain_grp = df[~df['ExitFlag']].groupby(
    ['JobInvolvement','PerformanceRating']).size().reset_index(name='n')
```

```
axes[0].scatter(retain_grp['JobInvolvement'],
                retain_grp['PerformanceRating'],
                s=retain_grp['n'] * 4,
                color='#378ADD', alpha=0.55,
                edgecolors='white', linewidth=1,
                label='Retain', zorder=3)

axes[0].scatter(exit_grp['JobInvolvement'],
                exit_grp['PerformanceRating'],
                s=exit_grp['n'] * 4,
                color='#E24B4A', alpha=0.75,
                edgecolors='white', linewidth=1,
                label='Exit', zorder=4)

# Annotate each exit bubble with count
for _, row in exit_grp.iterrows():
    axes[0].text(row['JobInvolvement'],
                 row['PerformanceRating'] + 0.12,
                 str(int(row['n'])),
                 ha='center', fontsize=9,
                 fontweight='bold', color='#E24B4A')

axes[0].axvline(2.5, color='#888', lw=1.5,
                linestyle=':', alpha=0.7, label='Cutoff')
axes[0].axhline(2.5, color='#888', lw=1.5,
                linestyle=':', alpha=0.7)

axes[0].text(1.2, 2.65, 'EXIT ZONE\n(low performance\n+ low engagement)',
             fontsize=9, color='#E24B4A', fontweight='bold',
             bbox=dict(boxstyle='round,pad=0.3',
                       facecolor='#fff0f0',
                       edgecolor='#E24B4A', linewidth=1))

axes[0].set_xlabel('Job Involvement (1=Low → 4=Very High)', fontsize=11)
axes[0].set_ylabel('Performance Rating (1=Low → 4=Outstanding)', fontsize=11)
axes[0].set_title('Bubble size = headcount per cell', fontsize=10, color='#666')
axes[0].set_xticks([1,2,3,4])
axes[0].set_yticks([1,2,3,4])
axes[0].legend(fontsize=10, markerscale=0.6)
axes[0].spines[['top','right']].set_visible(False)

# ── Chart 2: Stacked bar — cleaner story ──────────────
pivot = df.groupby(['JobInvolvement', 'ExitFlag']).size().unstack(fill_value=0)
pivot.columns = ['Retain', 'Exit']
pivot_pct = pivot.div(pivot.sum(axis=1), axis=0) * 100

bottoms = np.zeros(len(pivot_pct))
for label, color in [('Retain','#378ADD'), ('Exit','#E24B4A')]:
    vals = pivot_pct[label].values
    bars = axes[1].bar(pivot_pct.index.astype(str),
                       vals, bottom=bottoms,
                       color=color, label=label,
                       width=0.55, edgecolor='white', linewidth=1.5)
    for bar, v, b in zip(bars, vals, bottoms):
        if v > 6:
            axes[1].text(bar.get_x() + bar.get_width()/2,
                         b + v/2,
                         f'{v:.0f}%',
                         ha='center', va='center',
                         fontsize=11, fontweight='bold',
                         color='white')
    bottoms += vals

# Annotate headcount below
for i, (inv, row) in enumerate(pivot.iterrows()):
    axes[1].text(i, -5, f'n={int(row.sum())}',
                 ha='center', fontsize=9, color='#666')

axes[1].set_xlabel('Job Involvement Level', fontsize=11)
```

```
axes[1].set_ylabel('% of employees', fontsize=11)
axes[1].set_title('Exit % by Engagement Level\n(lower engagement → higher exit rate)',
                  fontweight='bold')
axes[1].legend(fontsize=10)
axes[1].set_ylim(-10, 110)
axes[1].spines[['top','right']].set_visible(False)

plt.tight_layout()
plt.savefig('performance_vs_engagement_fixed.png', dpi=150, bbox_inches='tight')
plt.show()
```




```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np
import seaborn as sns
from sklearn.ensemble          import RandomForestClassifier
from sklearn.linear_model      import LogisticRegression
from sklearn.model_selection   import train_test_split, cross_val_score
```